
LEARNING PIECEWISE-SMOOTH DYNAMICAL SYSTEMS

Davide Murari

Department of Applied Mathematics
and Theoretical Physics
University of Cambridge
dm2011@cam.ac.uk

Erik Jansson

Department of Applied Mathematics
and Theoretical Physics
University of Cambridge
eoj23@cam.ac.uk

Chris Budd OBE

Department of Mathematical Sciences
University of Bath
mascjb@bath.ac.uk

Carola-Bibiane Schönlieb

Department of Applied Mathematics
and Theoretical Physics
University of Cambridge
cbs31@cam.ac.uk

ABSTRACT

Discovering dynamical systems from trajectory data is a central problem in applied mathematics and engineering. Whilst recent advances in machine learning have led to strong progress in data-driven system identification, much less attention has been given to systems with discontinuous dynamics. These systems are nevertheless highly relevant in applications, including climate dynamics and mechanical systems with friction. In this work, we consider the problem of identifying piecewise-smooth dynamical systems directly from trajectory data. Compared with the smooth setting, this requires recovering the governing equations and detecting the switching hyperplanes that separate different dynamical regimes and characterising their behaviour, such as sliding motion. We present a modular framework for discovering such systems by first estimating switching hyperplanes from data and then learning smooth dynamics within each region using geometry-constrained neural networks. The geometry-learning phase is studied from a statistical perspective, analysing the identifiability of the discontinuities and the robustness of the procedure. We also introduce a novel neural network architecture with a prescribed discontinuity set, and provide a theoretical analysis of its approximation properties. The approach is tested on low-dimensional benchmark problems, including dry-friction oscillators and the PP04 climate model for the ice ages.

Keywords Data-driven modelling · Filippov systems · Discontinuous Neural Networks · Scientific machine learning

1 Introduction

Push a heavy box across a floor, and it does not move at once, until suddenly, it does. Open a stiff door slowly and the hinge does not turn smoothly but in tiny catches and releases. In both cases, the dynamics do not vary smoothly but jump between distinct regimes, sticking and slipping, as the system passes from one regime to another.

Systems like these are not unusual. We consider *piecewise-smooth systems*, whose vector fields are smooth within separate regions $\mathcal{P}_k$ of state space but may jump across the *switching hyperplanes* $\Sigma_{k,\ell}$ dividing them. Throughout this work, such systems are interpreted in the sense of Filippov, which replaces the discontinuous vector field on a switching hyperplane by a set-valued convexification of its nearby limiting values. We refer to systems equipped with this solution concept as *Filippov systems*. At a regular switching hyperplane, Filippov solutions may cross the hyperplane or slide along an attracting part of it [2, 14, 16, 20]. Discontinuous differential equations arise in mechanical systems with friction [22], sliding-mode control [15], robotics [1], and the glacial cycles of the Earth's climate [4, 27, 28].

Often the governing dynamics are unknown and must be recovered from observed trajectories. For piecewise-smooth systems, this requires learning the vector field in each regime, locating the switching hyperplanes, and characterising

the behaviour at those hyperplanes. A misplaced switching hyperplane misplaces the discontinuities, regardless of how accurately the local dynamics are learned. Figure 1 illustrates this setting for the PP04 climate model: the discontinuity is confined to an affine plane, and the learned model reproduces reference trajectories that repeatedly cross this plane.

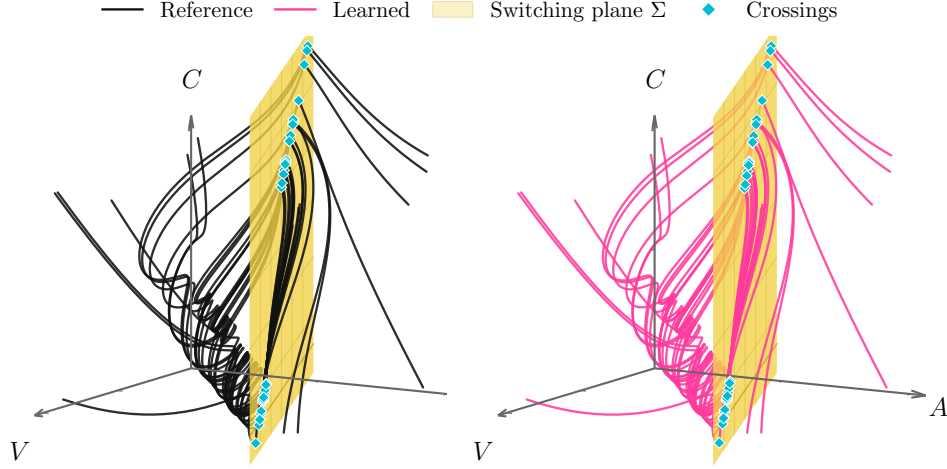


Figure 1: The PP04 system [4, 27] is a three-dimensional climate model of the ice ages. The vector field is discontinuous across the affine plane Σ , shown in yellow, and the cyan markers indicate trajectory crossings of this plane. The left panel shows trajectories of the reference system, while the right panel shows rollouts of the learned discontinuous model from the same initial conditions. The learned model is supplied with the true switching plane. The close agreement between the two trajectory families demonstrates its ability to reproduce the discontinuous dynamics. Further details are provided in Section 5.5.2.

Data-driven modelling of dynamical systems is commonly pursued in two complementary ways. One seeks to recover explicit governing equations from observations, deriving a symbolic approximation of them [3, 10, 12]. A related difficulty is that simple governing equations may only exist in appropriate coordinates. This has motivated autoencoder-based approaches that simultaneously learn latent coordinates and parsimonious dynamics [7, 26]. Another way to model dynamical systems instead learns the time evolution directly from trajectory data using neural or latent dynamical models. This includes recurrent architectures for time-series forecasting [19, 34], neural ordinary differential equations [9], and latent or controlled variants designed for partially observed or irregularly sampled trajectories [23, 31]. Universal differential equations further combine mechanistic model components with neural parameterisations of unknown terms [30].

Within scientific machine learning, an important refinement is to constrain the architecture so that the learned model respects the known structure of the dynamics. For Hamiltonian and Lagrangian systems, this has led to Hamiltonian neural networks, Lagrangian neural networks, symplectic networks, and symplectic neural flows, where conservation laws, constraints, or symplecticity are built into the parametrisation [5, 6, 11, 18, 21]. The present work follows the same structure-preserving philosophy but applies it to non-smooth systems. The relevant structure in our setting is the geometry of the switching set and the requirement that learned discontinuities remain confined to it. Existing work has considered learning local dynamics when the switching geometry is known, including friction problems [25]. Here, we additionally seek to recover that geometry from trajectory data.

Main contributions We introduce a geometry-first, two-stage framework for discovering piecewise-smooth dynamical systems from trajectory data. Our main contributions are three.

First, we develop a method for recovering arrangements of affine switching hyperplanes before fitting the vector field. Their sign patterns define interpretable polyhedral regions, see Figure 2, so the inferred switching geometry can be inspected and validated independently of the learned dynamics. We complement this method with a detectability analysis quantifying how jump size, observation noise, and derivative-estimation error affect geometry recovery.

Second, we introduce a geometry-preserving neural architecture whose discontinuities are confined, by construction, to a prescribed hyperplane arrangement. It can therefore learn distinct dynamics in adjacent regions without introducing discontinuities elsewhere. The regression experiments in Section 5.3 demonstrate this distinction directly.

Third, we prove that this architecture can approximate piecewise-smooth vector fields to arbitrary accuracy on compact sets away from the switching interfaces, while preserving the prescribed discontinuity set. Experiments on dry friction oscillators and the PP04 climate model validate both stages of the framework. In the reported baseline comparisons, the proposed models achieve lower rollout and vector-field errors than smooth Neural ODEs while using an order of magnitude fewer trainable parameters.

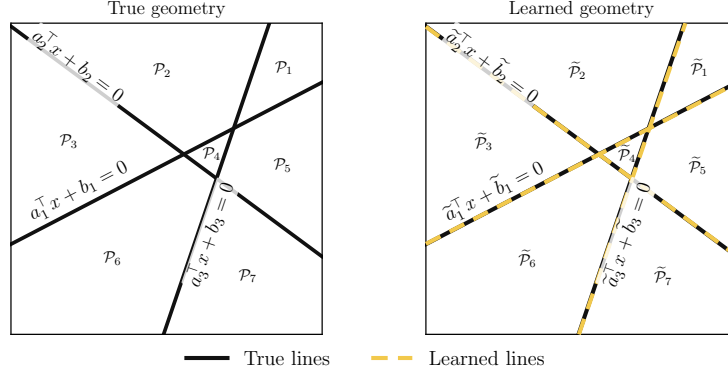


Figure 2: Schematic illustration of the geometry-learning stage. The true switching geometry is an arrangement of affine lines, which partitions the state space into polyhedral regions $\mathcal{P}_i$. From trajectory data, the algorithm estimates the switching geometry and obtains an approximate arrangement, shown on the right, with corresponding regions $\tilde{\mathcal{P}}_i$. More details on this experiment are in Appendix E.

Modularity and interpretability of the methodology The two stages of the framework are modular. The geometry-estimation stage is driven by *derivative approximations* from trajectory data, and the derivative-estimation procedure can be adapted to the noise level, sampling rate, and dimension of the problem. In this paper, this stage combines derivative-jump candidate extraction with random sample consensus (RANSAC) hyperplane fitting, followed by total least squares refinement. This stage can be skipped when the switching geometry is known in advance. The dynamics-learning stage can likewise incorporate prior knowledge about the local dynamics. For example, if each smooth piece is known to be affine, one can use a piecewise-affine model. If the local dynamics are Hamiltonian, one can use a Hamiltonian neural network [18] in each region. In our numerical experiments, we do not assume such detailed prior knowledge. Instead, we use a general piecewise-smooth neural vector field.

Outline of the paper This paper is structured as follows: The necessary background on piecewise-smooth dynamical systems is presented in Section 2. Section 3 describes the dataset and the joint learning of geometry and dynamics. Some theoretical aspects of the proposed methodology are considered in Section 4, including an analysis of the hyperplane detectability from noisy data and the approximation properties of the considered network. We include a selection of numerical experiments in Section 5 looking at friction and climate models. These experiments illustrate both the methodology and the theoretical results. Finally, we draw some conclusions from this work.

2 Piecewise-smooth dynamical systems

In this work, we use the term *piecewise-smooth system* to refer to a dynamical system whose vector field may be discontinuous across affine switching hyperplanes $\Sigma_{k,\ell}$ [16] and is continuously differentiable away from them. Its solutions are understood in the Filippov sense [14, 16]. We now make this setting precise.

We focus on systems with smoothness regions given by polyhedral partitions induced by finite arrangements of affine hyperplanes. More precisely, let $\{H_1, \dots, H_R\}$ be a finite affine hyperplane arrangement of $\mathbb{R}^d$, and let $P = \{\mathcal{P}_1, \dots, \mathcal{P}_K\}$ be the collection of connected components of

$$\mathbb{R}^d \setminus \bigcup_{r=1}^R H_r.$$

Then each $\mathcal{P}_k$, $k = 1, \dots, K$, is an open convex cell of the form

$$\mathcal{P}_k = \{x \in \mathbb{R}^d \mid A_k x < b_k\}, \quad A_k \in \mathbb{R}^{m_k \times d}, \quad b_k \in \mathbb{R}^{m_k}, \quad (1)$$

where the inequality is understood componentwise, and such that $\bigcup_{k=1}^K \overline{\mathcal{P}_k} = \mathbb{R}^d$, where $\overline{\mathcal{P}_k}$ denotes the closure of $\mathcal{P}_k$. For every pair of regions sharing a codimension-one boundary, we denote by $\Sigma_{k,\ell} := \text{aff}(\overline{\mathcal{P}_k} \cap \overline{\mathcal{P}_\ell})$ the affine hyperplane containing the facet shared by $\mathcal{P}_k$ and $\mathcal{P}_\ell$. We refer to $\Sigma_{k,\ell}$ as a *switching hyperplane*. We consider dynamical systems governed by piecewise- $\mathcal{C}^1$ vector fields $f : \mathbb{R}^d \times \mathbb{R} \rightarrow \mathbb{R}^d$, namely systems of the form

$$\dot{x} = f(x, t) := f_k(x, t), \quad x \in \mathcal{P}_k, \quad k = 1, \dots, K, \quad (2)$$

where each $f_k : \mathcal{P}_k \times \mathbb{R} \rightarrow \mathbb{R}^d$ is continuously differentiable. Thus, all discontinuities of f are confined to the shared facets, each of which lies in its corresponding switching hyperplane $\Sigma_{k,\ell}$. We denote the union of these facets by Σ , that is,

$$\Sigma := \mathbb{R}^d \setminus \bigcup_{k=1}^K \mathcal{P}_k = \bigcup_{r=1}^R H_r.$$

Throughout this work, we assume that each f_k admits a continuously differentiable extension $f_k : \mathbb{R}^d \times \mathbb{R} \rightarrow \mathbb{R}^d$, and we use the same notation for the vector field on $\mathcal{P}_k$ and its extension.

The values of f on Σ may be assigned arbitrarily without changing its Filippov regularisation: because the facets have Lebesgue measure zero, the regularisation depends on the neighbouring limiting vector fields rather than on pointwise values assigned on the facets. At regular points of codimension-one facets, this regularisation simplifies, as discussed in more detail below. At intersections of several facets, the regularisation accounts for the limiting vector fields from all adjacent regions. A Filippov solution is an absolutely continuous curve whose derivative belongs to the Filippov regularisation of f almost everywhere [14, 16].

A typical example is the case of two half-spaces separated by a hyperplane

$$\Sigma := \{x \in \mathbb{R}^d : s(x) = 0\}, \quad s(x) := a^\top x + b,$$

where $a \in \mathbb{R}^d$ satisfies $\|a\|_2 = 1$ and $b \in \mathbb{R}$, so that

$$\dot{x} = f(x, t) := \begin{cases} f^+(x, t), & s(x) > 0, \\ f^-(x, t), & s(x) < 0. \end{cases} \quad (3)$$

This single-hyperplane setting describes the local behaviour near a regular codimension-one facet separating two regions of a more general partition. For $x \in \Sigma$, the normal components of the two limiting vector fields are

$$\nu^+(x, t) := a^\top f^+(x, t), \quad \nu^-(x, t) := a^\top f^-(x, t).$$

If ν^+ and ν^- have the same sign, trajectories locally cross the switching hyperplane. If

$$\nu^+(x, t) < 0 < \nu^-(x, t),$$

then both limiting vector fields point towards Σ , and the hyperplane is locally attracting at (x, t) . In the Filippov convention [16], *sliding motion* on the locally attracting part of the hyperplane Σ is then described by the convex combination

$$f_\Sigma(x, t) = \alpha(x, t) f^+(x, t) + (1 - \alpha(x, t)) f^-(x, t),$$

where $\alpha \in [0, 1]$ is chosen so that the trajectory remains tangent to the hyperplane, i.e.,

$$a^\top f_\Sigma(x, t) = 0.$$

Equivalently,

$$\alpha(x, t) = \frac{\nu^-(x, t)}{\nu^-(x, t) - \nu^+(x, t)}.$$

This gives the sliding vector field along Σ . The opposite sign configuration,

$$\nu^-(x, t) < 0 < \nu^+(x, t),$$

corresponds to a locally repelling or escaping part of the hyperplane, where forward solutions starting on Σ may fail to be unique. Figure 3 illustrates the two behaviours that are most important for the learning problem: transversal crossings, which generate localised derivative jumps, and attracting sliding, where trajectories remain on the switching hyperplane and must be propagated with the Filippov sliding vector field f_Σ .

For further background, see [14, 16] and the references therein.

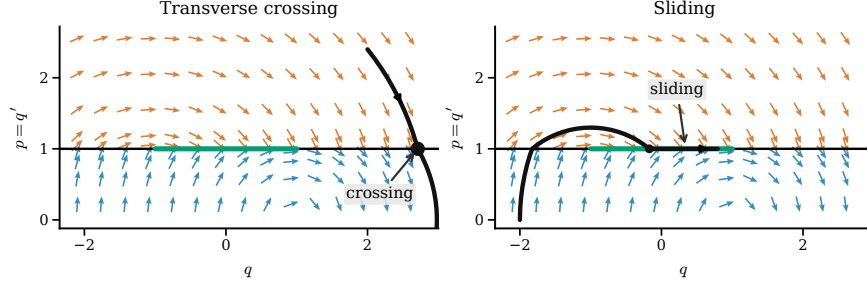


Figure 3: Basic switching behaviours in the toy Filippov system $\dot{q} = p$, $\dot{p} = -q + \text{sign}(1 - p)$. The switching hyperplane is $\Sigma = \{p = 1\}$, shown by the horizontal black line. The green segment marks the attracting sliding region $-1 < q < 1$. On this segment, the one-sided normal velocities satisfy $-q - 1 < 0 < -q + 1$, so the Filippov vector field is tangent to Σ and gives $\dot{q} = 1$, $\dot{p} = 0$. The black curves show representative trajectories: one crosses Σ transversally outside the sliding interval, while the other reaches the attracting part of Σ and then slides along it.

3 Learning piecewise-smooth dynamical systems

Our approach to learning piecewise-smooth dynamical systems is split into two sequential tasks.

First, we estimate the switching geometry by recovering an arrangement of affine switching hyperplanes. The sign patterns of this arrangement define the approximate polyhedral regions

$$\tilde{P} = \{\tilde{P}_1, \dots, \tilde{P}_{\tilde{K}}\},$$

which are used during dynamics learning.

Second, we learn a piecewise-smooth vector field adapted to this recovered partition.

This procedure separates “geometry learning” from “dynamics learning”, and allows inductive biases specific to each task to be introduced independently.

We now describe the core building blocks of our methodology, starting with the dataset, then moving to the geometry and dynamics learning phases.

3.1 Dataset

We consider N reference trajectories and their noisy observations, indexed by $n = 1, \dots, N$. The n th trajectory starts from the initial condition x_0^n at time t_0^n , and is sampled at the regularly spaced times

$$t_j^n := t_0^n + j h, \quad j = 0, \dots, J,$$

with final time $T_n = t_j^n = t_0^n + Jh$. For simplicity, all trajectory segments are taken to have the same number of time steps. The extension to segments of non-uniform length is immediate. Let $x(t; t_0, x_0)$ denote the solution satisfying

$$\dot{x}(t; t_0, x_0) = f(x(t; t_0, x_0), t), \quad x(t_0; t_0, x_0) = x_0.$$

For the n th trajectory, we write

$$x_*^n(t) = x(t; t_0^n, x_0^n), \quad t \in [t_0^n, T_n].$$

We assume that the sampling times are known exactly and are not subject to timing errors. In the idealised data model, the noise-free states $x_*^n(t_j^n)$ are additively perturbed by independent Gaussian observation noise. The observations are thus

$$y_j^n := x_*^n(t_j^n) + \eta_j^n, \quad j = 0, \dots, J, \quad (4)$$

where $\eta_j^n \sim \mathcal{N}(0, \sigma^2 I_d)$ are independent across both n and j , $I_d \in \mathbb{R}^{d \times d}$ denotes the identity matrix, and $\sigma \geq 0$ is the noise level. For numerically generated data, the discrepancy from the exact solution may also contain time-discretisation errors. The analysis in Section 4.1 isolates the effect of Gaussian observation noise. In the numerical experiments, see Section 5, the reference trajectories are generated synthetically from the benchmark systems. When an exact solution is available between switching events, we use it together with event localisation at switching times; otherwise, we use an event-aware numerical solver. The system-specific sampling boxes, time grids, and solver details are given in Section 5.2.

3.2 Learning the geometry

There are two natural signals for recovering Σ : (i) trajectory points lying on sliding segments, and (ii) jumps in estimated velocities across consecutive samples. The first signal is present only when sliding occurs and is highly sensitive to noise, which can break collinearity. We therefore base the geometry-learning stage on the second signal: large changes in estimated velocity are treated as candidate discontinuity points.

Implementing velocity-jump-based geometry learning requires choosing a velocity estimator. In principle, our method is independent of this choice, but its performance depends on the properties of the adopted approach. For instance, in the regime of negligible observational noise, we can compute velocity estimates along trajectory n at the points y_j^n using central finite differences, i.e.,

$$v_{h,j}^n = \frac{y_{j+1}^n - y_{j-1}^n}{2h}, \quad j = 1, \dots, J-1, \quad (5)$$

meaning that each trajectory provides $J-1$ finite-difference estimates.

If noise is present, we can instead employ a noise-mitigating Savitzky–Golay velocity estimator [32, 33], so that

$$v_{h,j}^n = \sum_{k=-r}^r s_k y_{j+k}^n, \quad j = r, \dots, J-r, \quad (6)$$

where the coefficients are obtained by fitting a polynomial of degree p to uniformly spaced coordinate indices by linear least squares and therefore do not depend on the observed data. Nonlinear alternatives include total-variation regularised differentiation [8, 33]. Our numerical tests use Savitzky–Golay differentiation as a shared geometry-learning procedure across the experiments because it provides accurate hyperplane recovery at modest computational cost.

We then compute the velocity jump $\Delta v_{h,j}^n = v_{h,j+1}^n - v_{h,j}^n$. Note that Equation (5) and Equation (6) are *linear* estimators, that is, they are of the form

$$v_{h,j}^n = \sum_{k=-r}^r w_k y_{j+k}^n \quad (7)$$

for some cut-off $r \in \mathbb{N}$ and weights $w_k \in \mathbb{R}$, $k = -r, \dots, r$. Linear estimators are inexpensive and easy to analyse, but can be sensitive to observational noise, as illustrated in Section 5.4.

Algorithm overview. Algorithm 1 states the geometry-learning procedure for a single dominant affine switching hyperplane, which is the setting used in the main experiments. Having computed the jumps, we rank all the derivative-jump magnitudes globally and retain the largest fraction $\rho \in (0, 1)$. For each retained jump interval, we also include neighbouring intervals, since a true crossing may affect several adjacent velocity estimates. The endpoints of these intervals form an overcomplete candidate cloud $\mathcal{C}$. This cloud contains points near the switching hyperplane, but also outliers caused by curvature, noise, and derivative-estimation artefacts. We proceed with the hyperplane recovery only if there are enough candidates, i.e., $|\mathcal{C}| \geq \max\{3, m_{\min}, d\}$ with $m_{\min} = 30$ for the single-hyperplane recovery tasks. We therefore fit the hyperplane using random sample consensus (RANSAC) [17], a robust randomised fitting procedure that repeatedly proposes a hyperplane from a minimal subset of points, counts the number of candidate points within a prescribed distance tolerance, and keeps the hyperplane with the largest consensus set. We then refine the selected inliers by total least squares. See Appendix A for more background on RANSAC.

Remark 3.1. This stage requires the observed trajectories to produce velocity jumps that dominate smooth curvature and derivative-estimation error. Transversal crossings and transitions into or out of sliding can provide such a signal, whereas grazing contacts and trajectories confined to one side of the switching hyperplane may not. For example, in $\dot{x} = \text{sign}(y)$, $\dot{y} = 0$, the vector field is discontinuous across $\Sigma = \{y = 0\}$, but every trajectory has constant y and therefore produces no derivative jump. Thus, identifiability requires both a sufficiently large jump and sufficient interaction between the observed trajectories and Σ . We analyse the jump-to-noise balance in Section 4.1 and test it numerically in Section 5.

A simple extension for recovering multiple switching hyperplanes is to apply the procedure sequentially, removing inliers associated with the strongest hyperplane recovered before fitting the next one. This produces an arrangement of recovered affine interfaces, rather than an explicit reconstruction of the polygonal cells, and is intended for well-separated interfaces. We illustrate this sequential extension and a support-based estimate of the number of hyperplanes in Appendix E.

Algorithm 1 Single-hyperplane recovery

Require: Warmup trajectories $\{y_j^n\}_{n=1, j=0}^{N, J} \subset \mathbb{R}^d$; step size h ; Savitzky–Golay window w and polynomial degree p ; retained jump fraction $\rho \in (0, 1)$ and neighbouring interval radius R ; RANSAC tolerance ε ; RANSAC iterations M ; minimum support $m_{\min}$.

Ensure: Unit normal $a \in \mathbb{R}^d$ and offset $b \in \mathbb{R}$ defining the recovered switching hyperplane $\Gamma = \{x \in \mathbb{R}^d : a^\top x + b = 0\}$.

- 1: Estimate velocities with Savitzky–Golay differentiation:
 $\hat{v}_j^n \leftarrow \text{SG}_{w,p}[y^n]_j$, for $n = 1, \dots, N$ and $j = 0, \dots, J$.
- 2: Compute derivative-jump magnitudes
 $J_n(j) \leftarrow \|\hat{v}_{j+1}^n - \hat{v}_j^n\|_2$, for $j = 0, \dots, J-1$.
- 3: Rank all jump intervals (n, j) globally by $J_n(j)$.
- 4: Set $L \leftarrow NJ$.
- 5: Keep the largest $\max\{1, \text{round}(\rho L)\}$ jump intervals.
- 6: Add the immediate neighbouring intervals of each retained jump:
 $\mathcal{E} \leftarrow \{(n, \ell) : \ell \in \{j-R, \dots, j, \dots, j+R\}, 0 \leq \ell < J, (n, j) \text{ is retained}\}$.
- 7: Build the candidate cloud from both endpoints of every selected interval:
 $\mathcal{C} \leftarrow \{y_\ell^n, y_{\ell+1}^n : (n, \ell) \in \mathcal{E}\}$.
- 8: Remove duplicate points from $\mathcal{C}$.
- 9: Set $m_{\text{req}} \leftarrow \max\{3, m_{\min}, d\}$.
- 10: **if** $|\mathcal{C}| < m_{\text{req}}$ **then**
- 11: **fail.**
- 12: **end if**
- 13: $(a_\star, b_\star, \mathcal{I}_\star) \leftarrow \text{RANSAC}(\mathcal{C}, \varepsilon, M, m_{\text{req}})$. ▷ For a description of RANSAC see Appendix A.
- 14: **if** RANSAC fails **then**
- 15: **fail.**
- 16: **end if**
- 17: Fit (a, b) by total least squares on $\mathcal{I}_\star$. ▷ $\mathcal{I}_\star$ is the inlier set of RANSAC.
- 18: Normalise (a, b) so that $\|a\|_2 = 1$ and fix a deterministic sign convention.
- 19: Recompute the refined consensus set
 $\mathcal{I} \leftarrow \{x \in \mathcal{C} : |a^\top x + b| \leq \varepsilon\}$.
- 20: **if** $|\mathcal{I}| < m_{\text{req}}$ **then**
- 21: **fail.**
- 22: **end if**
- 23: **return** (a, b) .

3.3 Learning the dynamics

Once an approximate geometry has been obtained, we need to approximate the dynamics. In this work, geometry recovery and dynamics learning are performed sequentially: after estimating the polyhedral partition P , we fix it during the training of the vector field. If the switching geometry is known from modelling considerations, the geometry-learning stage can be skipped, and the same dynamics-learning procedure can be applied directly on the prescribed partition.

We denote the recovered partition by $\tilde{P} = \{\tilde{\mathcal{P}}_1, \dots, \tilde{\mathcal{P}}_{\tilde{K}}\}$, allowing its number of regions $\tilde{K}$ to differ from that of the target partition K . In our experiments, we assume we know the number of hyperplanes. However, in Appendix E, we show that such a number can be detected during the geometry-learning phase. The goal now is to define an approximate vector field of the form

$$\dot{x} = \tilde{f}(x, t) := \tilde{f}_k(x, t), \quad x \in \tilde{\mathcal{P}}_k, \quad k = 1, \dots, \tilde{K}.$$

Depending on the available structural knowledge, we choose a class $\mathcal{F}_{\tilde{P}}^d$ for $\tilde{f}$. We call this class *adapted to the partition* $\tilde{P}$ if every element has discontinuities only on the hyperplanes defining the partition $\tilde{P}$. This prevents the learned vector field from creating spurious switching hyperplanes away from the recovered geometry. For example, if the vector field is known to be autonomous and piecewise affine, we could set

$$\mathcal{F}_{\tilde{P}}^d = \left\{ \tilde{f} : \mathbb{R}^d \rightarrow \mathbb{R}^d \mid \tilde{f}(x) = \sum_{k=1}^{\tilde{K}} 1_{\tilde{\mathcal{P}}_k}(x)(B_k x + c_k), \quad B_k \in \mathbb{R}^{d \times d}, \quad c_k \in \mathbb{R}^d \right\}.$$

When lacking further structural knowledge on the target dynamics, we model the local maps $\tilde{f}_k$ using sufficiently flexible neural networks. In Section 4.2, we show how to realise such partition-adapted neural vector fields without explicitly evaluating indicator functions.

Once an approximation space is defined, we use the available data to recover a vector field that reproduces the behaviour observed in the training data. To do so, we minimise $\mathcal{L}_{\text{traj}}(g)$ over $g \in \mathcal{F}_{\tilde{P}}^d$, where

$$\mathcal{L}_{\text{traj}}(g) = \frac{1}{NJd} \sum_{n=1}^N \sum_{j=1}^J \|y_j^n - \varphi_g(t_j^n; t_0^n, \xi_n)\|^2, \quad (8)$$

$\varphi_g(t; t_0, \xi)$ denotes a numerical approximation at time t of the corresponding solution of

$$\dot{z}(\tau) = g(z(\tau), \tau), \quad z(t_0) = \xi.$$

To improve the training process, we increase the length of the trajectory segments J gradually during training. We call this curriculum training and provide more details in Section 5. Here we set $\xi_n = y_0^n$, the noisy observed initial condition. The numerical method φ that we consider is described in Section 5, and the precise class of neural vector fields that we use is defined in Section 4.

In some numerical experiments, we augment the trajectory-matching loss above with velocity-based auxiliary terms. Let $\mathcal{I}$ denote the set of trajectory indices (n, j) considered by the auxiliary losses. Let $v_{h,j}^n$ denote a velocity estimate computed from the observed trajectory using one of the linear estimators in Equations (5) to (7). We define

$$\mathcal{L}_{\text{vf}}(g) = \frac{1}{d|\mathcal{I}_{\text{vf}}|} \sum_{(n,j) \in \mathcal{I}_{\text{vf}}} \|g(y_j^n, t_j^n) - v_{h,j}^n\|^2, \quad (9)$$

where $\mathcal{I}_{\text{vf}} \subseteq \mathcal{I}$ denotes the set of samples used for vector-field supervision and $|\mathcal{I}_{\text{vf}}|$ the number of such samples. This set may exclude samples close to the switching set, where velocity estimates are least reliable.

The numerical experiments using these directional regularisers have a single switching hyperplane with unit normal a . For arrangements of several hyperplanes, analogous penalties could be computed for each hyperplane and averaged. Let

$$\Pi_{\text{tan}} = I_d - aa^\top$$

be the orthogonal projection onto the tangent space of the hyperplane, and define

$$\mathcal{I}_{\text{nor}} = \{(n, j) \in \mathcal{I} : |v_{h,j}^n \cdot a| > \tau_{\text{nor}}\}, \quad \mathcal{I}_{\text{tan}} = \{(n, j) \in \mathcal{I} : \|\Pi_{\text{tan}} v_{h,j}^n\|_2 > \tau_{\text{tan}}\}.$$

The normal regulariser penalises a predicted normal component that points in the opposite direction to the estimated velocity,

$$\mathcal{L}_{\text{nor}}(g) = \frac{1}{|\mathcal{I}_{\text{nor}}|} \sum_{(n,j) \in \mathcal{I}_{\text{nor}}} \max\{0, -(g(y_j^n, t_j^n) \cdot a) \text{sign}(v_{h,j}^n \cdot a)\}, \quad (10)$$

while the tangential regulariser matches the tangential components,

$$\mathcal{L}_{\text{tan}}(g) = \frac{1}{d|\mathcal{I}_{\text{tan}}|} \sum_{(n,j) \in \mathcal{I}_{\text{tan}}} \|\Pi_{\text{tan}} g(y_j^n, t_j^n) - \Pi_{\text{tan}} v_{h,j}^n\|^2. \quad (11)$$

The general objective used in the numerical experiments is therefore

$$\mathcal{J}(g) = \lambda_{\text{traj}} \mathcal{L}_{\text{traj}}(g) + \lambda_{\text{vf}} \mathcal{L}_{\text{vf}}(g) + \lambda_{\text{nor}} \mathcal{L}_{\text{nor}}(g) + \lambda_{\text{tan}} \mathcal{L}_{\text{tan}}(g), \quad (12)$$

with the weights, thresholds, and active terms specified in Section 5.

4 Theoretical analysis

We now analyse certain theoretical aspects of the methodology. First, we study how observational noise interplays with the properties of jumps across the switching hyperplane in the context of geometry learning, and then we study the approximation properties of a class of piecewise-smooth neural networks later used in the experiments.

4.1 Analysis of geometry learning

In this section, we address some theoretical aspects of the geometry learning described in Section 3.2. Here, we consider the simplified case where $\mathbb{R}^d$ is separated by one hyperplane, i.e., we suppose that

$$\Sigma := \{x \in \mathbb{R}^d : s(x) = 0\}, \quad s(x) := a^\top x + b,$$

where $a \in \mathbb{R}^d \setminus \{0\}$ and $b \in \mathbb{R}$, and the goal is to recover a and b .

Velocity-based discontinuity learning with linear velocity estimation methods may be sensitive to noise. Since the observations are affected by random noise, they are random variables, and consequently so are the velocity estimates. As linear velocity estimators rely on numerical differentiation, they amplify high-frequency components of the data, such as noise. The detection of Σ also relies on locally amplifying the velocity jump. If the noise magnitude is too high, a linear velocity estimator cannot detect jumps across the boundary. The amplification has a floor that no choice of weights can beat. For an estimator $v_{h,j}^n = \sum_{k=-r}^r w_k y_{j+k}^n$ applied to data of the form (4), the random part $\sum_{k=-r}^r w_k \eta_{j+k}^n$ is a linear combination of independent Gaussians, so

$$v_{h,j}^n - \sum_{k=-r}^r w_k x_*^n(t_{j+k}^n) \sim N(0, \sigma^2 \|w\|_2^2 I_d), \quad \|w\|_2^2 = \sum_{k=-r}^r w_k^2.$$

The consistency conditions $\sum_k w_k = 0$ and $\sum_k w_k k h = 1$ require, by Cauchy–Schwarz,

$$1 = \left| \sum_{k=-r}^r w_k k h \right| \leq h \sqrt{\sum_{k=-r}^r w_k^2} \sqrt{\sum_{k=-r}^r k^2} = h \|w\|_2 \sqrt{S_r}, \quad S_r = \frac{r(r+1)(2r+1)}{3},$$

so $\|w\|_2 \geq 1/(h\sqrt{S_r})$. This floor decreases as the window radius r grows, but increases as the step size h shrinks. It explains why a Savitzky–Golay estimator is more robust to noise than central differences, and also why it will still struggle when σ is too large.

The noise floor on a single velocity estimate has consequences for which jumps across Σ are detectable. The next result combines this floor with the smoothing error incurred by the choice of window size r , addressing which jump sizes are detectable. To this end, consider the estimated velocity jump

$$\Delta v_{h,j}^n = v_{h,j+1}^n - v_{h,j}^n = \sum_{k=-r}^r w_k y_{j+1+k}^n - \sum_{k=-r}^r w_k y_{j+k}^n = \sum_{k=-r}^{r+1} c_k y_{j+k}^n,$$

where

$$c_k = \begin{cases} -w_{-r} & \text{if } k = -r, \\ w_r & \text{if } k = r+1, \\ w_{k-1} - w_k & \text{else.} \end{cases}$$

If the velocity estimator satisfies the same consistency conditions $\sum_k w_k = 0$, $\sum_k w_k k h = 1$, then $\sum_{k=-r}^{r+1} c_k = 0$ and $\sum_{k=-r}^{r+1} c_k k h = 0$, i.e., no jump is detected on constant and constant-velocity trajectories. Further, it holds that $\|c\|_2 \leq 2\|w\|_2$. We are now ready to state the proposition.

Proposition 4.1. *Consider the velocity estimator $v_{h,j}^n = \sum_{k=-r}^r w_k y_{j+k}^n$ satisfying the consistency conditions $\sum_k w_k = 0$, $\sum_k w_k k h = 1$. Let f^+ and f^- be smooth and extendable to the whole domain. Assume that a reference trajectory x_*^n crosses Σ transversally at one single time τ , and that without loss of generality, $s(x) < 0$ before the crossing. Set $p = x_*^n(\tau) \in \Sigma$. Denote by x^+ the solution to $\dot{x} = f^+(x, t)$, with initial time τ and starting value $x^+(\tau) = p$, and denote by x^- the trajectory of f^- agreeing with x_*^n before the switch. By continuity, $x^-(\tau) = x^+(\tau) = p$. The velocity jump is given by*

$$f^+(p, \tau) - f^-(p, \tau) = \kappa e, \quad \kappa > 0, \quad e \in \mathbb{R}^d,$$

where $\kappa = \|f^+(p, \tau) - f^-(p, \tau)\|_2$ and $\|e\|_2 = 1$. Assume that the second derivatives of both x^+ and x^- are uniformly bounded by M . Let

$$R = \sum_{k=-r}^{r+1} c_k \max(t_{j+k} - \tau, 0), \quad \alpha = M \sum_{k: t_{j+k} > \tau} |c_k| (t_{j+k} - \tau)^2, \quad \beta = \frac{M}{2} \sum_{k=-r}^{r+1} |c_k| (t_{j+k} - t_j)^2.$$

Consider now two scenarios. In the first, the trajectory jumps in the stencil used to compute $\Delta v_{h,j}^n$, and in the second, it follows x^- throughout. For simplicity, denote the first jump estimate by Δ_1 , and the second by Δ_2 . Then,

1. Both Δ_1 and Δ_2 are Gaussian, that is, $\Delta_i \sim \mathcal{N}(\mu_i, C)$, with

$$C = \sigma^2 \|c\|_2^2 I_d, \quad \mu_1 - \mu_2 = \kappa R e + \varrho,$$

where $\varrho \in \mathbb{R}^d$ satisfies $\|\varrho\|_2 \leq \alpha$.

2. The expected magnitudes satisfy

$$\mathbb{E}[\|\Delta_1\|_2] \geq \kappa|R| - \beta - \alpha, \quad \mathbb{E}[\|\Delta_2\|_2] \leq \beta + \sigma\|c\|_2\sqrt{d},$$

so the expected jump magnitude exceeds the expected smooth magnitude whenever $\kappa|R| > 2\beta + \alpha + \sigma\|c\|_2\sqrt{d}$.

The constant R measures how much of the post-crossing displacement the jump stencil captures. The constants α and β are second-derivative Taylor remainders, both controlled by M : β bounds the spurious jump produced by a smooth trajectory of nonzero acceleration, while α bounds the error in approximating the post-crossing displacement by its leading linear-in-time term. Detectability therefore requires the scaled jump $\kappa|R|$ to dominate α , β , and the noise term $\sigma\|c\|_2\sqrt{d}$.

Proof. As above, the randomness is introduced by the estimator being applied to the observational noise. Due to the linearity, the resulting quantity is also normally distributed, that is,

$$\sum_{k=-r}^{r+1} c_k \eta_{j+k}^n \sim \mathcal{N}(0, \sigma^2 \|c\|_2^2 I_d).$$

Consider now Δ_2 , i.e., when the stencil contains no jumps. In that case, we Taylor expand and write

$$x^-(t_{j+k}) = x^-(t_j) + kh\dot{x}^-(t_j) + r_k,$$

where $r_k = \int_{t_j}^{t_{j+k}} (t_{j+k} - s)\ddot{x}^-(s) ds$, so that $\|r_k\|_2 \leq \frac{M}{2}(t_{j+k} - t_j)^2$. The annihilation property of the velocity jump estimator means that

$$\mu_2 = \sum_{k=-r}^{r+1} c_k r_k,$$

which satisfies

$$\|\mu_2\|_2 \leq \frac{M}{2} \sum_{k=-r}^{r+1} |c_k| (t_{j+k} - t_j)^2 = \beta.$$

The case when there is a jump is similar. The deterministic trajectory is $x^-(t) + (x^+(t) - x^-(t))\mathbf{1}_{t>\tau}$, where $\mathbf{1}_{t>\tau}$ is 1 if $t > \tau$ and 0 elsewhere. Since $x^+(t) - x^-(t)$ is smooth, we Taylor expand around τ and obtain

$$x^+(t_{j+k}) - x^-(t_{j+k}) = \kappa e(t_{j+k} - \tau) + \varrho_k,$$

where $\varrho_k = \int_{\tau}^{t_{j+k}} (\ddot{x}^+(s) - \ddot{x}^-(s))(t_{j+k} - s) ds$. Since $\|\ddot{x}^+ - \ddot{x}^-\|_2 \leq 2M$, we see that $\|\varrho_k\|_2 \leq M(t_{j+k} - \tau)^2$. Summing this up, we obtain

$$\mu_1 = \mu_2 + \sum_{k:t_{j+k}>\tau} c_k [\kappa e(t_{j+k} - \tau) + \varrho_k] = \kappa e R + \sum_{k:t_{j+k}>\tau} c_k \varrho_k,$$

where we have that

$$\left\| \sum_{k:t_{j+k}>\tau} c_k \varrho_k \right\|_2 \leq M \sum_{k:t_{j+k}>\tau} |c_k| (t_{j+k} - \tau)^2 = \alpha.$$

For the second statement, first note that by the triangle inequality and Jensen's inequality,

$$\mathbb{E}[\|\Delta_2\|_2] \leq \|\mu_2\|_2 + \sigma\|c\|_2\sqrt{d} \leq \beta + \sigma\|c\|_2\sqrt{d}.$$

For the second bound, Jensen's inequality using the norm function, followed by the reverse triangle inequality, gives

$$\mathbb{E}[\|\Delta_1\|_2] \geq \|\mathbb{E}[\Delta_1]\|_2 \geq \kappa|R| - \beta - \alpha.$$

□

In practice, this means that the gap between the jump magnitude and the noise standard deviation must be sufficiently large to distinguish smooth points from true jumps, and that sensitivity decreases as the dimension grows. We numerically test the balance between the discontinuity gap and the noise magnitude in Section 5.

Remark 4.1. The proposition above is a local signal-detection result. It assumes that the crossing time τ lies in the range of the velocity-jump stencil and gives a condition under which the resulting derivative jump is distinguishable from smooth curvature and observation noise. The subsequent hyperplane-fitting step uses the corresponding neighbouring trajectory points as candidates for Σ . This introduces a separate conditioning issue. Points with small signed distance to Σ need not be tightly localised around the crossing time if the normal crossing velocity is small.

Indeed, let $p = x(\tau)$ and suppose, for simplicity, that $\Sigma = \{x : s(x) = 0\}$ is a hyperplane with $s(x) = a^\top x + b$ and $\|a\|_2 = 1$. For a nearby time t on either side of the crossing,

$$s(x(t)) = (t - \tau)a^\top f^{\text{pre/post}}(p, \tau) + \mathcal{O}(|t - \tau|^2).$$

Thus, the signed-distance signal scales like $|t - \tau|v_{\text{nor}}$, where

$$v_{\text{nor}} = \min \{ |a^\top f^{\text{pre}}(p, \tau)|, |a^\top f^{\text{post}}(p, \tau)| \}.$$

With observations $y(t) = x(t) + \eta(t)$ and $\eta(t) \sim \mathcal{N}(0, \sigma^2 I_d)$, one has

$$s(y(t)) - s(x(t)) = a^\top \eta(t) \sim \mathcal{N}(0, \sigma^2).$$

Consequently, the time interval over which the clean signed distance is comparable to the observation noise has a width of order σ/v_{nor} . Indeed, this is the regime where $|t - \tau|v_{\text{nor}} \approx \sigma$, so the observed signed distance $s(y(t))$ is strongly affected by noise and is no longer a reliable proxy for the clean distance to Σ . When v_{nor} is small, a larger portion of the trajectory near the crossing is ambiguous from the point of view of geometric localisation. Thus, the jump-to-noise condition above should be read together with a non-grazing condition: the normal crossing velocity must be large enough, relative to the sampling step and observation noise, for the candidate cloud to localise Σ reliably.

4.2 Universal neural network for piecewise-smooth dynamical systems

Let us fix a polyhedral partition $P = \{\mathcal{P}_1, \dots, \mathcal{P}_K\}$ of $\mathbb{R}^d$ induced by a finite affine hyperplane arrangement. We now define a neural class $\mathcal{F}_P^d$ adapted to this partition. The class has two key properties: its discontinuities are confined to the prescribed arrangement, and it can approximate piecewise-smooth vector fields arbitrarily well almost everywhere on compact sets.

Let $\text{ReLU}(x) = \max\{0, x\}$ denote the ReLU function, and $\mathcal{H}(x)$ the Heaviside step function. We use the convention $\mathcal{H}(r) = 1$ for $r \geq 0$ and $\mathcal{H}(r) = 0$ for $r < 0$, applied componentwise to vectors. For a vector $J \in \mathbb{R}^k$, we define the nonlinear function $\text{ReLU}_J : \mathbb{R}^k \rightarrow \mathbb{R}^k$ by $\text{ReLU}_J(x) = \text{ReLU}(x) + J \odot \mathcal{H}(x)$, as in [13]. For fixed $d_1, d_2 \in \mathbb{N}$, we introduce the networks

$$\begin{aligned} \mathcal{F}_{\text{ReLU}_J}^{d_1, d_2} &:= \left\{ \mathcal{N} : \mathbb{R}^{d_1} \rightarrow \mathbb{R}^{d_2} \mid \mathcal{N} = W_L \circ \text{ReLU}_{J_L} \circ \dots \circ W_1 \circ \text{ReLU}_{J_1} \circ W_0, \right. \\ &\quad L \in \mathbb{N}, W_\ell : \mathbb{R}^{h_\ell} \rightarrow \mathbb{R}^{h_{\ell+1}} \text{ affine}, \ell = 0, \dots, L \\ &\quad \left. J_\ell \in \mathbb{R}^{h_\ell}, \ell = 1, \dots, L, h_0 = d_1, h_{L+1} = d_2, h_1, \dots, h_L \in \mathbb{N} \right\}. \end{aligned}$$

We denote by $\mathcal{F}_{\text{ReLU}}^{d_1, d_2} \subset \mathcal{F}_{\text{ReLU}_J}^{d_1, d_2}$ the subclass for which all jump parameters vanish, i.e., the set of deep ReLU feedforward networks. In this section, we repeatedly use the identities

$$\mathcal{H}(x) = [I_d \quad -I_d] \text{ReLU}_J \left(\begin{bmatrix} I_d \\ I_d \end{bmatrix} x \right), \quad x \in \mathbb{R}^d, J = \begin{bmatrix} 1_d \\ 0_d \end{bmatrix} \in \mathbb{R}^{2d}, \quad (13)$$

and

$$x = [I_d \quad -I_d] \text{ReLU}_J \left(\begin{bmatrix} I_d \\ -I_d \end{bmatrix} x \right), \quad x \in \mathbb{R}^d, J = \begin{bmatrix} 0_d \\ 0_d \end{bmatrix} \in \mathbb{R}^{2d}. \quad (14)$$

In particular, $\mathcal{H} \in \mathcal{F}_{\text{ReLU}_J}^{d, d}$ and the identity map belongs to $\mathcal{F}_{\text{ReLU}}^{d, d}$.

Lemma 4.1 (Closure under affine combinations and parallel concatenation). *For every choice of dimensions, both $\mathcal{F}_{\text{ReLU}_J}^{d_1, d_2}$ and $\mathcal{F}_{\text{ReLU}}^{d_1, d_2}$ are closed under multiplication with a scalar, linear combination, and parallel concatenation.*

The proof follows similar ideas as in [29, Section 5.1], and it can be found in Appendix B.

Adopting the notation in (1), we set

$$d_P := \sum_{k=1}^K m_k, \quad A_P := \begin{bmatrix} A_1 \\ \vdots \\ A_K \end{bmatrix}, \quad b_P := \begin{bmatrix} b_1 \\ \vdots \\ b_K \end{bmatrix}, \quad T_P(x) := b_P - A_P x \in \mathbb{R}^{d_P},$$

and define $z_P(x) = \mathcal{H}(T_P(x)) \in \{0, 1\}^{d_P}$,

$$\Gamma_P := \bigcup_{\ell=1}^{d_P} \{x \in \mathbb{R}^d : (T_P(x))_\ell = 0\}.$$

Thus z_P is locally constant on $\mathbb{R}^d \setminus \Gamma_P$.

Definition 4.1 (Geometry-preserving class associated with P). For the fixed state dimension d , we define $\mathcal{F}_P^d$ as the set of maps

$$(x, t) \mapsto \Psi(x, t, \Phi(z_P(x)))$$

such that

$$\Phi \in \mathcal{F}_{\text{ReLU}_J}^{d_P, d_1}, \quad \Psi \in \mathcal{F}_{\text{ReLU}}^{1+d+d_1, d}, \quad d_1 \in \mathbb{N}.$$

Remark 4.2. Time-independent maps of the form $x \mapsto \Psi(x, \Phi(z_P(x)))$ with $\Psi \in \mathcal{F}_{\text{ReLU}}^{d+d_1, d}$, $\Phi \in \mathcal{F}_{\text{ReLU}_J}^{d_P, d_1}$, $d_1 \in \mathbb{N}$, can also be realised as maps in $\mathcal{F}_P^d$. If $\mathcal{N} \in \mathcal{F}_P^d$ is time-independent, we write $\mathcal{N}(x)$ and avoid including the variable t for conciseness.

Proposition 4.2 (No new jumps). *Every element of $\mathcal{F}_P^d$ is continuous on $(\mathbb{R}^d \setminus \Gamma_P) \times \mathbb{R}$.*

Proof. Let $\mathcal{N} \in \mathcal{F}_P^d$. If $x_0 \notin \Gamma_P$, then z_P is locally constant near x_0 . Therefore, near x_0 , the map $x \mapsto \Phi(z_P(x))$ is constant, while $(x, t) \mapsto \Psi(x, t, \Phi(z_P(x)))$ is the composition of continuous maps. Thus $\mathcal{N}$ is continuous at (x_0, t) for every $t \in \mathbb{R}$. $\square$

Proposition 4.3 (Linearity of the geometry-preserving class). *$\mathcal{F}_P^d$ is a vector space over $\mathbb{R}$.*

The proof follows from the vector space property of $\mathcal{F}_{\text{ReLU}_J}^{d_1, d_2}$. We include it in Appendix B.

Theorem 4.1 (Exact ReLU gating on compact sets). *Let $\Omega \subset \mathbb{R}^d$ be compact, $T > 0$, and $R \in \mathcal{F}_{\text{ReLU}}^{d+1, m}$. Then there exists $\tilde{R} \in \mathcal{F}_{\text{ReLU}}^{d+2, m}$ such that*

$$\tilde{R}(x, t, s) = \begin{cases} R(x, t), & s = 1, \\ 0, & s = 0, \end{cases} \quad \forall (x, t, s) \in \Omega \times [0, T] \times \{0, 1\}.$$

Since affine maps of the form $(x, t) \mapsto Ax + b \in \mathbb{R}^m$ belong to $\mathcal{F}_{\text{ReLU}}^{d+1, m}$, the result above applies to affine time-independent maps as well.

Proof. Since R is continuous, and $[0, T]$ and Ω are compact,

$$\lambda := \sup_{\substack{x \in \Omega \\ t \in [0, T]}} \|R(x, t)\|_\infty < \infty.$$

We propagate the binary variable s unchanged through the network while computing $R(x, t)$ in parallel. Since $s \in \{0, 1\} \subset [0, \infty)$, one has $\text{ReLU}(s) = s$, so this can be done using only ReLU layers. At the final stage, define

$$\tilde{R}(x, t, s) := \text{ReLU}(R(x, t) - \lambda(1-s)1_m) - \text{ReLU}(-R(x, t) - \lambda(1-s)1_m).$$

If $s = 1$, then

$$\tilde{R}(x, t, 1) = \text{ReLU}(R(x, t)) - \text{ReLU}(-R(x, t)) = R(x, t).$$

If $s = 0$, then for every component one has $|R_j(x, t)| \leq \lambda$, hence $\tilde{R}(x, t, 0) = 0$ as desired. $\square$

We now show that the networks in $\mathcal{F}_P^d$ can approximate every piecewise-smooth vector field within an arbitrary accuracy.

Definition 4.2 (Piecewise-smooth vector field on a fixed arrangement). Let $\Omega \subset \mathbb{R}^d$ be compact and let $P = \{\mathcal{P}_1, \dots, \mathcal{P}_K\}$ be as above. A function $F : \Omega \times \mathbb{R} \rightarrow \mathbb{R}^d$ is said to be piecewise-smooth with respect to P if for every $k \in \{1, \dots, K\}$ there exists a map

$$F_k : \overline{\Omega \cap \mathcal{P}_k} \times \mathbb{R} \rightarrow \mathbb{R}^d$$

which is continuous on $\overline{\Omega \cap \mathcal{P}_k} \times \mathbb{R}$ and smooth on $(\Omega \cap \mathcal{P}_k) \times \mathbb{R}$, and such that

$$F(x, t) = F_k(x, t), \quad \forall x \in \Omega \cap \mathcal{P}_k, t \in \mathbb{R}.$$

Theorem 4.2 (Approximation of piecewise-smooth systems without creating new jumps). Let $\Omega \subset \mathbb{R}^d$ be compact, $T > 0$, and $P = \{\mathcal{P}_1, \dots, \mathcal{P}_K\}$ a finite family of pairwise disjoint open convex polyhedra with

$$\Omega \setminus \Gamma_P = \bigcup_{k=1}^K (\Omega \cap \mathcal{P}_k).$$

Let $F : \Omega \times \mathbb{R} \rightarrow \mathbb{R}^d$ be piecewise smooth with respect to P . Then for every $\varepsilon > 0$, there exists $\mathcal{N} \in \mathcal{F}_P^d$ such that

$$\sup_{\substack{x \in \Omega \setminus \Gamma_P \\ t \in [0, T]}} \|F(x, t) - \mathcal{N}(x, t)\|_\infty < \varepsilon.$$

Proof. For every $k = 1, \dots, K$, we have that $F_k : \overline{\Omega \cap \mathcal{P}_k} \times [0, T] \rightarrow \mathbb{R}^d$ is continuous. Since $\overline{\Omega \cap \mathcal{P}_k} \times [0, T]$ is compact, the universal approximation theory of ReLU networks, see [35], ensures that there exists $\tilde{\Psi}_k \in \mathcal{F}_{\text{ReLU}}^{d+1, d}$ such that

$$\sup_{\substack{x \in \overline{\Omega \cap \mathcal{P}_k} \\ t \in [0, T]}} \|F_k(x, t) - \tilde{\Psi}_k(x, t)\|_\infty < \varepsilon.$$

Consider the projection $\Pi_k : \mathbb{R}^{d_P} \rightarrow \mathbb{R}^{m_k}$ onto the block corresponding to $\mathcal{P}_k$, so that

$$\Pi_k T_P(x) = b_k - A_k x.$$

We define the function $\chi_k(z) = \mathcal{H}(1_{m_k}^\top \Pi_k z - m_k + \frac{1}{2})$ in $\mathcal{F}_{\text{ReLU}, J}^{d_P, 1}$, compare [24, Equation 1]. If $x \notin \Gamma_P$, then $x \in \mathcal{P}_k$ if and only if all entries of $\Pi_k z_P(x) = \mathcal{H}(b_k - A_k x)$ are equal to 1. Therefore, $1_{\mathcal{P}_k}(x) = \chi_k(z_P(x))$ for $x \in \Omega \setminus \Gamma_P$.

By Theorem 4.1, there exists a ReLU network $\Psi_k \in \mathcal{F}_{\text{ReLU}}^{d+2, d}$ such that

$$\Psi_k(x, t, s) = \begin{cases} \tilde{\Psi}_k(x, t), & s = 1, \\ 0, & s = 0, \end{cases} \quad \forall (x, t, s) \in \Omega \times [0, T] \times \{0, 1\}.$$

Therefore the map

$$\mathcal{N}_k(x, t) := \Psi_k(x, t, \chi_k(z_P(x)))$$

belongs to $\mathcal{F}_P^d$ and satisfies

$$\mathcal{N}_k(x, t) = 1_{\mathcal{P}_k}(x) \tilde{\Psi}_k(x, t), \quad \forall (x, t) \in (\Omega \setminus \Gamma_P) \times [0, T].$$

Since $\mathcal{F}_P^d$ is a vector space, the sum

$$\mathcal{N}(x, t) := \sum_{k=1}^K \mathcal{N}_k(x, t)$$

belongs to $\mathcal{F}_P^d$. Moreover, for $(x, t) \in (\Omega \setminus \Gamma_P) \times [0, T]$,

$$\mathcal{N}(x, t) = \sum_{k=1}^K 1_{\mathcal{P}_k}(x) \tilde{\Psi}_k(x, t).$$

For each $x \in \Omega \setminus \Gamma_P$, there is a unique $\underline{k} \in \{1, \dots, K\}$ such that $x \in \mathcal{P}_{\underline{k}}$. Therefore,

$$\|\mathcal{N}(x, t) - F(x, t)\|_\infty = \|\tilde{\Psi}_{\underline{k}}(x, t) - F(x, t)\|_\infty < \varepsilon$$

as desired. $\square$

Remark 4.3 (Exact representation of piecewise-affine vector fields on compacts). Given a partition $P = \{\mathcal{P}_1, \dots, \mathcal{P}_K\}$ and a compact set $\Omega \subset \mathbb{R}^d$ as in Theorem 4.2, every piecewise-affine function

$$x \mapsto \sum_{k=1}^K 1_{\mathcal{P}_k}(x)(W_k x + w_k), \quad W_1, \dots, W_K \in \mathbb{R}^{d \times d}, \quad w_1, \dots, w_K \in \mathbb{R}^d$$

is the restriction over Ω of a function $\mathcal{N} \in \mathcal{F}_P^d$. Such a result follows by picking $\tilde{\Psi}_k(x, t) = W_k x + w_k$ in the proof above.

Remark 4.4 (Extended approximation spaces). Replacing the raw time input by a continuous feature map $\varphi(t)$, or augmenting Ψ with other continuous features of x and t , does not enlarge the possible discontinuity set because $z_P(x)$ remains the only discontinuous input. In the periodically forced experiments, we therefore use maps of the form

$$(x, t) \mapsto \Psi(x, \varphi(t), \Phi(z_P(x))),$$

where

$$\Psi \in \mathcal{F}_{\text{ReLU}}^{d+2k+d_1, d}, \quad \Phi \in \mathcal{F}_{\text{ReLU}_J}^{d_P, d_1},$$

and

$$\varphi(t) = [\cos(\omega_1 t + \theta_1) \quad \sin(\omega_1 t + \theta_1) \quad \dots \quad \cos(\omega_k t + \theta_k) \quad \sin(\omega_k t + \theta_k)]^\top \in \mathbb{R}^{2k}.$$

The frequencies $\omega_1, \dots, \omega_k > 0$ may be fixed from prior knowledge or chosen as random Fourier features.

5 Numerical experiments

We now test the methodology introduced and analysed above. The experiments address three questions: (1) whether the proposed architecture confines discontinuities to a prescribed partition, (2) whether derivative-jump RANSAC can recover the switching geometry from the training trajectories, and (3) whether the recovered geometry supports accurate trajectory learning. We study the geometry and dynamics learning phases both in isolation, in Section 5.3 and Section 5.4, and when combined, in Section 5.5. We also test the effect of incorporating additional prior knowledge, such as the second-order structure of the target dynamics and the forcing frequency in periodically forced systems. Unless stated otherwise, all experiments based on training a network use the random seed 7.

In the experiments in which we recover the geometry from trajectories, we focus on systems with one dominant affine switching surface. The dry-friction and the climate-based PP04 benchmarks both have a physically meaningful single switch, and this setting isolates the interaction between derivative-jump recovery and dynamics learning without adding the separate task of assigning several recovered facets to regions. The approximation architecture is formulated for general finite polyhedral partitions, and the known-geometry regression experiment already tests a multi-region partition. The more general case is discussed in Appendix E, which gives a synthetic demonstration of sequential RANSAC–TLS recovery for multiple affine interfaces.

The GitHub repository associated with the paper is at <https://github.com/davidemurari/learn-piecewise-smooth>.

Benchmark dynamical systems The three benchmark systems are chosen to test different parts of the methodology.

Benchmark A: We first isolate the dynamics-learning architecture on a two-dimensional piecewise-linear oscillator with prescribed switching geometry. The vector field is

$$F(x) = \begin{bmatrix} p \\ -q - \alpha(q)p \end{bmatrix}, \quad \alpha(q) = \begin{cases} 0.9, & q < -1, \\ -0.9, & -1 \leq q < 1, \\ 0.9, & q \geq 1. \end{cases} \quad (15)$$

This example has three smooth regions separated by the two switching lines $q = \pm 1$, and is used to test whether the proposed architecture can approximate a discontinuous vector field without introducing spurious jumps away from the prescribed partition.

We then test the complete learning pipeline on two families of trajectory-learning problems: dry-friction oscillators and the PP04 system.

Benchmark B: The dry-friction oscillator. This system, also considered in [25] and analysed in [2, 14], models the stick-slip dynamics of a forced mass moving on a surface with friction, and is useful because it combines a simple

switching geometry with non-trivial Filippov behaviour. It allows us to separate three effects: the benefit of knowing the switching geometry, the benefit of imposing the mechanical second-order structure, and the degradation caused by noisy trajectory observations. The system has state $x = (q, p)$ and stick-slip dynamics given by

$$\dot{q} = p, \quad \dot{p} = -q - cp + A \cos(\omega t) - \mu \operatorname{sign}(p), \quad (16)$$

with switching hyperplane $\Sigma = \{(q, p) : p = 0\}$. We consider two forced cases with $c = 0.1$, $\mu = 0.5$, and $A = 1$, but different forcing frequencies as in [25]:

$$\omega = 0.3 \quad (\text{case 1}), \quad \omega = 0.15 \quad (\text{case 2}).$$

Benchmark C: The PP04 system. This climate-based system models the periodic behaviour of the ice ages [27, 28] and provides a complementary three-dimensional benchmark. In contrast to the dry-friction oscillator, its switching surface is an affine plane not aligned with a coordinate axis, and the system is periodically forced. The state is $x = (V, A, C)$ representing the global ice volume V , the Antarctic ice cover A and the atmospheric carbon dioxide level C . In this model, carbon dioxide is gradually absorbed by the oceans over a period of about 100k years, leading to global cooling and a growth of the ice sheets. At a critical concentration, the oceans can no longer absorb the carbon dioxide, and it is suddenly released into the atmosphere. This causes a sudden rise in C and rapid global warming, leading to a drop in V and A . The process then repeats, synchronised by the periodic variations in the solar insolation caused by the Milankovitch cycles. With the parameter values used in the experiments, the linear switching function (representing the oceanic stratification linked to the carbon dioxide absorption) is given by:

$$s(V, A, C) = aV - bA + d, \quad a = 0.3, \quad b = 0.7, \quad d = 0.27.$$

The switching surface (corresponding to the critical values of the parameters when the carbon dioxide is suddenly released from the oceans) is the plane in the 3-dimensional phase-space given by:

$$\Sigma = \{(V, A, C) : s(V, A, C) \equiv (a, -b, 0)^T (V, A, C) + d = 0\}.$$

Figure 1 visualises this plane in the (V, A, C) phase space and compares representative trajectories of the reference PP04 system with learned-model rollouts from the same initial conditions. Setting the (assumed periodic) solar insolation to $I_{65}(t) = \mu \sin(\omega t)$ (which varies according to the Milankovitch cycles [27]), the vector field (where time t is expressed in kilo-years) is

$$\begin{aligned} \dot{V} &= \frac{-xC - yI_{65}(t) + z - V}{\tau_V}, \\ \dot{A} &= \frac{V - A}{\tau_A}, \\ \dot{C} &= \frac{\alpha I_{65}(t) - \beta V + \gamma H(-s(V, A, C)) + \delta - C}{\tau_C}. \end{aligned} \quad (17)$$

Here the Heaviside function is defined by $H(z) = 1$ for $z \geq 0$ and $H(z) = 0$ otherwise. The γ term is active when $s(V, A, C) \leq 0$, including on the switching surface. In the trajectory-learning experiments all PP04 coefficients are fixed at $x = 1.3$, $y = 0.5$, $z = 0.8$, $\alpha = 0.15$, $\beta = 0.5$, $\gamma = 0.7$, $\delta = 0.4$, $\tau_V = 15$, $\tau_A = 12$, $\tau_C = 5$, $\mu = 0.467$, and $\omega = 0.1532$. The geometry-recovery diagnostic below instead varies γ while keeping every other coefficient and the switching plane fixed. This example tests whether the derivative-jump geometry-learning stage can accurately recover a non-axis-aligned switching plane for subsequent trajectory learning.

5.1 Evaluation metrics

For the different experiments, we isolate different aspects of the learning process. This is done by changing the architecture and its training regimen, as well as considering different evaluation metrics, which we introduce below.

Discontinuity detection diagnostic We define a metric to detect discontinuities and test it on the system in Equation (15). We describe it for this system, but it naturally extends.

For a learned vector field $\hat{F}$, output component k , grid point x , we compute

$$D_{h,k}(x) = \max_{v \in V} \left| \hat{F}_k(x + hv) - \hat{F}_k(x - hv) \right|, \quad V = \left\{ (1, 0), (0, 1), \frac{(1, 1)}{\sqrt{2}}, \frac{(1, -1)}{\sqrt{2}} \right\}.$$

For a continuous piecewise-linear function, $D_{h,k}(x)$ scales approximately linearly with h , so $D_{2h,k}(x)/D_{h,k}(x) \approx 2$. We retain a persistent jump score $P_{h,k}(x) = D_{h,k}(x)$ only where

$$\frac{D_{2h,k}(x)}{D_{h,k}(x) + 10^{-12}} < 1.35$$

and $D_{h,k}(x)$ exceeds a small amplitude floor. The floor is chosen componentwise as

$$\max \{10^{-6}, 10^{-2} \text{percentile}_{99}(D_{h,k})\}.$$

We then define the known-switch band

$$B_\delta = \{|q+1| \leq \delta\} \cup \{|q-1| \leq \delta\}, \quad \delta = 2h,$$

and compute the persistent-jump mass outside this band. For component k we define

$$M_k^{\text{tot}} = \sum_{x_j} P_{h,k}(x_j), \quad M_k^{\text{off}} = \sum_{x_j \notin B_\delta} P_{h,k}(x_j),$$

summing over a diagnostic grid. The off-switch fraction is $\rho_k^{\text{off}} = M_k^{\text{off}}/M_k^{\text{tot}}$, if $M_k^{\text{tot}} > 0$.

Rollout and vector field metrics Across the examples, the tables report trajectory-level and vector field-level diagnostics. Given a fixed set of N initial conditions, the rollout error is

$$\text{RMSE}_{\text{rollout}} = \sqrt{\frac{1}{N(K+1)} \sum_{n=1}^N \sum_{k=0}^K \|\hat{x}_n(t_k) - x_n(t_k)\|_2^2},$$

and the final-time error is

$$\text{RMSE}_{\text{final}} = \sqrt{\frac{1}{N} \sum_{n=1}^N \|\hat{x}_n(t_K) - x_n(t_K)\|_2^2}.$$

These two quantities measure whether the learnt dynamics extrapolate beyond the short training windows.

We also report vector field errors on a regular grid in the state domain, averaged over forcing phases when the system is non-autonomous:

$$\text{RMSE}_f = \sqrt{\frac{1}{M} \sum_{m=1}^M \|\hat{f}(x_m, t_m) - f(x_m, t_m)\|_2^2}.$$

The same quantity is also evaluated in a narrow band around the switching set. This banded error is a strict local diagnostic of how well the switching dynamics are captured.

The evaluation protocol uses 128 trajectories of 200 steps for each trajectory-learning case. Non-autonomous evaluation trajectories use independently sampled initial forcing phases. Vector-field diagnostics use an 80×80 state grid for dry friction and a 24^3 grid for PP04. Each non-autonomous grid is averaged over 16 uniformly spaced forcing phases. The switching band used for the local vector-field metric has half-width 0.05. Rollout metrics for the discontinuous dry-friction and PP04 models use the single-hyperplane Filippov solver described in Appendix D; continuous Neural ODE baselines use RK4.

Quality of the recovered switching geometry When the switching geometry is learned, the tables report the angle and offset errors of the recovered hyperplane. For a learned hyperplane $a^\top x + b = 0$ and the true hyperplane $a_\star^\top x + b_\star = 0$, both coefficient pairs are post-processed before evaluation. The normals are rescaled to unit Euclidean norm and oriented so that their inner product is non-negative. The offsets are rescaled and, when necessary, sign-flipped with their corresponding normals. Denote the resulting representations by $(\hat{a}, \hat{b})$ and $(\hat{a}_\star, \hat{b}_\star)$. The reported angle error, in degrees, is

$$\theta = \frac{180}{\pi} \arccos(\hat{a}^\top \hat{a}_\star),$$

and the offset error is

$$e_b = |\hat{b} - \hat{b}_\star|.$$

Both errors are therefore independent of the scaling and sign used to represent either hyperplane. For known-geometry runs, these quantities are zero by construction.

5.2 Trajectory data generation

For each discontinuous-model trajectory experiment, we generate two independent sets of trajectories. The main set is used for dynamics learning and is split into 90% training and 10% validation trajectories. Each dry-friction run uses a main set of 1,000 trajectories, while each PP04 run uses 5,000. The warm-up set contains the same number of trajectories as the corresponding main set and is used only to recover the switching geometry when that geometry is not supplied. The continuous baselines use the same main trajectory sets and splits, but do not generate a warm-up set.

We sample uniformly the initial states from $[-3, 3] \times [-2, 2]$ for dry friction and from $[0, 1]^3$ for PP04. The initial time t_0 is sampled uniformly over one forcing period for each trajectory.

The sampled trajectory segment is

$$\{x(t_0), x(t_0 + h), \dots, x(t_0 + kh)\}.$$

For forced dry friction, reference trajectories are generated by a dedicated event-aware integrator. It applies RK4 within each smooth branch, localises detected crossings of $p = 0$ by bisection, and explicitly handles attractive sticking intervals on the switching manifold. For PP04, the true flow is propagated exactly between events using the matrix exponential and a sinusoidal particular solution, see [27, Section 6.2]. Detected switching-plane crossings are localised by bisection before the remainder of the step is propagated on the new side.

Dry-friction main segments use $h = 0.01$ and $k = 10$, and warm-up segments use $h_{\text{warm}} = 0.01$ and $k_{\text{warm}} = 50$. PP04 main segments use $h = 0.05$ and $k = 50$, and warm-up segments use $h_{\text{warm}} = 0.02$ and $k_{\text{warm}} = 100$.

Time supplied to the learned vector field The learned discontinuous networks do not receive raw time as an input. Their smooth branch is evaluated as

$$\Psi(x, \Phi(z(x)), \tau(t))$$

when switching geometry is present, and as $\Psi(x, \tau(t))$ for a continuous model. With a known forcing frequency, $\tau(t) = (\cos(\omega t), \sin(\omega t))$. In the unknown-frequency experiment, $\tau(t)$ contains 16 fixed random Fourier frequencies and phases, represented by their sine and cosine pairs. The universality result in Theorem 4.2 persists within the family of piecewise-smooth vector fields, depending on t only through $\tau(t)$.

5.3 Known-geometry regression and discontinuity support

We isolate the role of the architectural prior in the dynamics-learning problem by considering a two-dimensional piecewise affine vector field for which the switching geometry is known exactly, see Equation (15). Both models are given the true switching code, and we ask whether the learned vector field introduces additional output jumps away from the prescribed switching set.

The switching set is given by:

$$\Sigma = \{(q, p) \in \mathbb{R}^2 : q = -1\} \cup \{(q, p) \in \mathbb{R}^2 : q = 1\},$$

which separates the state space into the three open polyhedra

$$\mathcal{P}_1 = \{q < -1\}, \quad \mathcal{P}_2 = \{-1 < q < 1\}, \quad \mathcal{P}_3 = \{q > 1\}.$$

Equivalently, the implementation uses the binary switching code

$$z_P(x) = \mathcal{H} \left(\begin{bmatrix} 1 & 0 \\ 1 & 0 \end{bmatrix} x + \begin{bmatrix} 1 \\ -1 \end{bmatrix} \right),$$

so that the three regions correspond to the codes $(0, 0)$, $(1, 0)$, and $(1, 1)$, respectively.

Architectures The first model is the time-independent version of the geometry-preserving class in Definition 4.1,

$$\mathcal{N}(x) = \Psi(x, \Phi(z_P(x))), \quad \Phi \in \mathcal{F}_{\text{ReLU}_J}^{2, d_1}, \quad \Psi \in \mathcal{F}_{\text{ReLU}}^{2+d_1, 2}.$$

The two switching hyperplanes are fixed to their true values. We refer to this model as the geometry-preserving architecture.

We also compare against an unstructured jump network. This model keeps the same fixed switching code, but concatenates it directly with the state and passes the result through a jump-ReLU network,

$$\tilde{\mathcal{N}}(x) = G(x, z_P(x)), \quad G \in \mathcal{F}_{\text{ReLU}_J}^{4, 2}.$$

This baseline therefore has access to the same exact partition information, but it does not use the separated Φ - Ψ parametrisation of Definition 4.1. The widths are chosen so that the two models have comparable size.

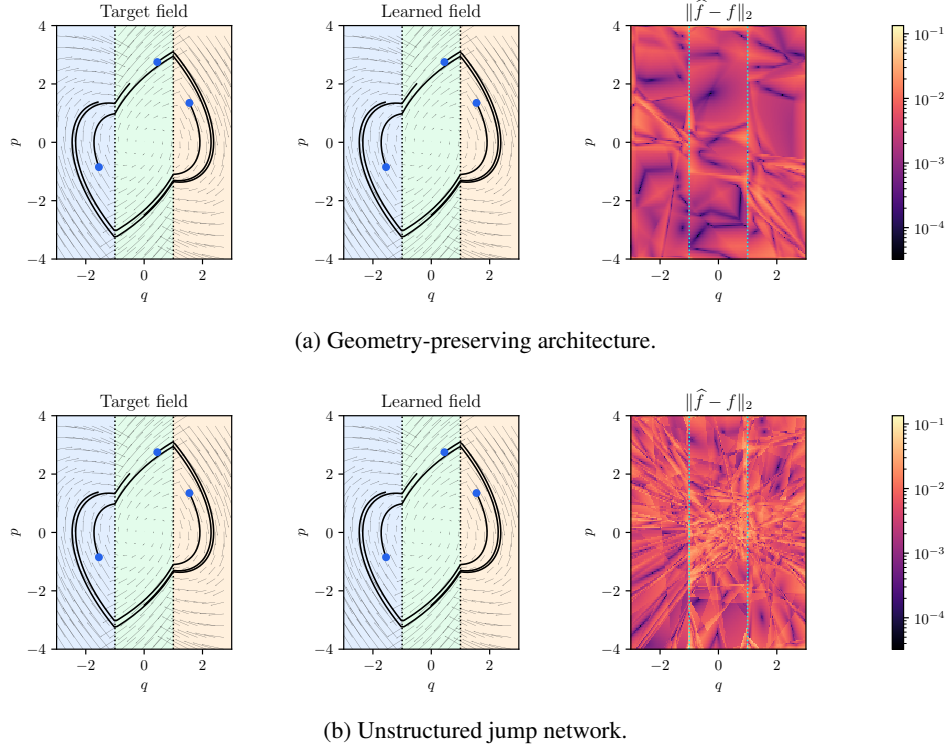


Figure 4: Qualitative reconstruction of the target vector field from noisy regression data. In each row, the first panel shows the target field, the second panel shows the learned field, and the third panel shows the pointwise vector error $\|\hat{f} - f\|_2$ on a logarithmic scale. Both architectures recover the main phase portrait and give small clean-grid errors, but the geometry-preserving architecture gives a smoother and more faithful reconstruction of the target partition structure. The displayed state domain is the full training rectangle $[-3, 3] \times [-4, 4]$, which contains every point of the diagnostic trajectories.

Training protocol Training data are generated directly from the vector field rather than from finite-difference trajectory estimates

$$x_n \sim \mathcal{U}([-3, 3] \times [-4, 4]), \quad y_n = F(x_n) + \eta_n, \quad \eta_n \sim N(0, \sigma^2 I_2), \quad \sigma = 0.03,$$

for $n = 1, \dots, N$. The model parameters are fitted by minimising the empirical mean-square regression loss

$$\mathcal{L}(\mathcal{N}) := \frac{1}{2N} \sum_{n=1}^N \|\mathcal{N}(x_n) - y_n\|_2^2.$$

Each model is trained for 1,000 epochs on 10,000 noisy uniformly sampled points and selected using an independent noisy validation set of 2,000 uniformly sampled points. A separate clean test set contains 6,000 uniformly sampled points. The geometry-preserving model uses one width-32 hidden layer in Φ and three width-32 hidden layers in Ψ . The unstructured model uses three hidden layers of width 45. The widths are chosen to give comparable model sizes: the geometry-preserving and unstructured models contain 4,482 and 4,592 trainable parameters, respectively. Both are optimised with Adam, weight decay 10^{-6} , gradient-norm clipping at 1, and a one-cycle learning-rate schedule from 10^{-4} to $5 \cdot 10^{-3}$ and back down by a final division factor of 10^4 . The checkpoint with the smallest validation mean-square error is restored for all reported diagnostics.

For the orbit diagnostic, both the target and learned fields are propagated with classical RK4 on the same uniform output grid. An outer RK4 step is recursively bisected into consecutive half-steps whenever the discrete region labels at its proposed endpoints differ.

Results As shown in Figure 4, both models recover the main phase portrait from noisy observations. The geometry-preserving model gives a more faithful field reconstruction and more closely reproduces the diagnostic trajectories

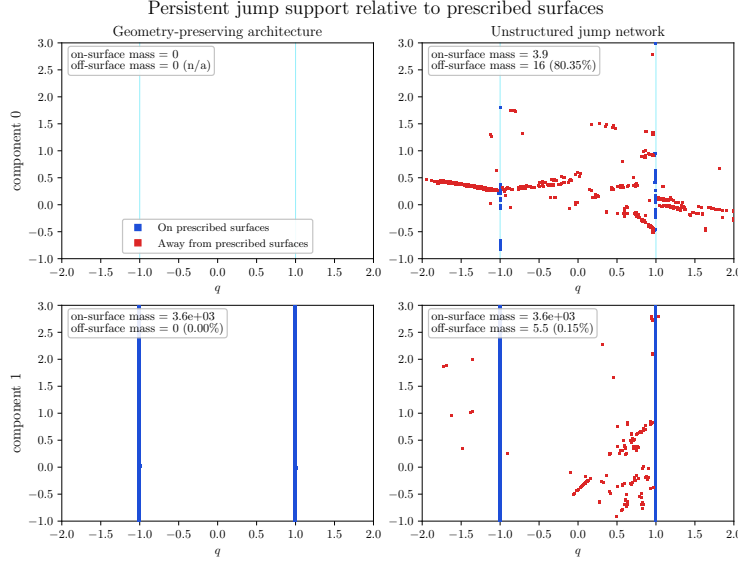


Figure 5: Persistent jump support after training on noisy regression targets with $\sigma = 0.03$. Blue points lie within a $2h$ neighbourhood of the prescribed surfaces, and red points lie outside it. The architecture fixes the potential discontinuity surfaces but does not prescribe which output components may jump across them. No persistent jump mass is detected in the geometry-preserving model’s first component, while all detected second-component mass lies on the prescribed surfaces. The unstructured jump network produces persistent support away from those surfaces. Pale cyan lines in the first row mark potential architectural surfaces. The stronger solid guides in the second row mark the target component’s actual switching surfaces.

throughout the training domain. The persistent-jump diagnostic in Figure 5 shows the central architectural difference. All detected persistent jumps produced by the geometry-preserving model remain confined to the prescribed switching surfaces, whereas the unstructured network also produces persistent jumps away from them. Thus, access to the correct partition alone is not sufficient to preserve its geometry. The constrained architecture enforces the intended support for discontinuities while retaining an accurate approximation of the dynamics.

5.4 Trajectory-based recovery of the discontinuities

This subsection focuses on the geometry learning phase. We consider the hyperplane learning procedure described in Algorithms 1 and 2. Figure 6 confirms that the candidate cloud of high-jump points that we work with after derivative estimation is overcomplete. Large derivative jumps may come from true discontinuity crossings, but also from smooth high-curvature motion, endpoint effects in the derivative estimate, or observation noise. Consequently, fitting a hyperplane by total least squares (TLS) to all candidates is unreliable. Instead, we use RANSAC to find a hyperplane supported by a coherent subset of candidates, and then refit it to the inliers using TLS.

Because both estimators receive exactly the same complete three-dimensional candidate array, the comparison isolates the effect of robust consensus selection. RANSAC selects the best-supported raw affine proposal, using the mean inlier distance only to break ties in support, and then applies TLS to the winning consensus. This remains appropriate only when a coherent switching trace is identifiable in the candidate cloud. The comparison shows that robust consensus selection is decisive at mild noise but cannot recover an identifiable plane once the switching trace is not sufficiently distinguishable from the effects of noise.

We also empirically validate the detectability result in Section 4.1. We consider the PP04 geometry-recovery problem, varying the coefficient γ which controls the magnitude of the jump across the discontinuity hyperplane. The observation-noise level remains $\sigma = 0.05$. Each success fraction is taken over 100 RANSAC random seeds for one paired trajectory/noise realisation. As shown in Figure 7, recovery becomes reliable once the discontinuity is sufficiently pronounced relative to the observation noise, although the transition is not strictly monotone. We remark that changing γ also changes the trajectories and the resulting candidate cloud. Therefore, this test supports the theoretical analysis, but considers a different dynamical system for each γ .

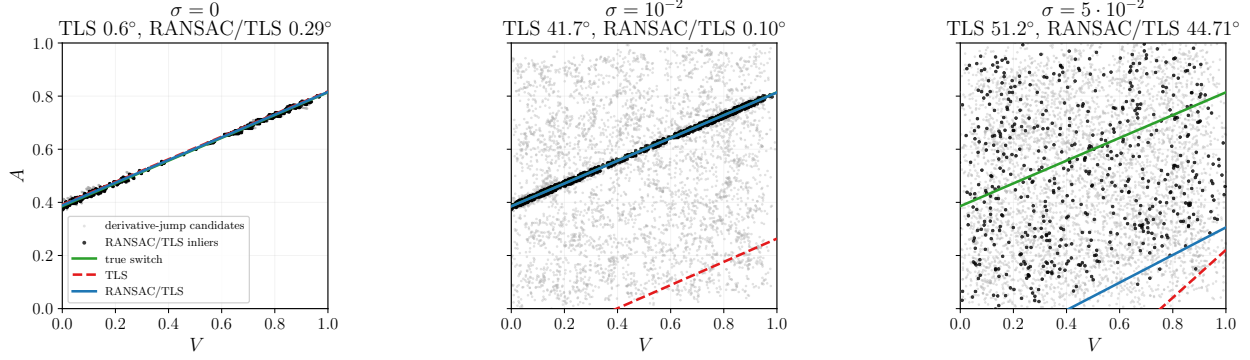


Figure 6: PP04 geometry recovery from derivative-jump candidates. Grey points are the overcomplete candidate cloud formed from the endpoints of the globally retained high-derivative-jump intervals after neighbour expansion, and black points are the final RANSAC/TLS inliers. Ordinary TLS and RANSAC receive the same candidate cloud. Green is the true switch, red dashed is ordinary TLS, and blue is RANSAC followed by a final TLS refinement on its consensus set. The panels correspond to observation-noise levels $\sigma = 0$, 10^{-2} , and 5×10^{-2} . The clean case shows that ordinary TLS works when the candidate cloud is already concentrated near the switch, the low-noise case shows the benefit of robust consensus selection in a contaminated cloud, and the high-noise case shows the recovery failure regime. At $\sigma = 0$, the TLS and RANSAC angle errors are approximately 0.60° and 0.29° . At $\sigma = 10^{-2}$, TLS fails with error 41.7° , whereas RANSAC recovers the plane with error 0.10° . At $\sigma = 5 \cdot 10^{-2}$, both methods fail, with errors 51.2° and 44.7° .

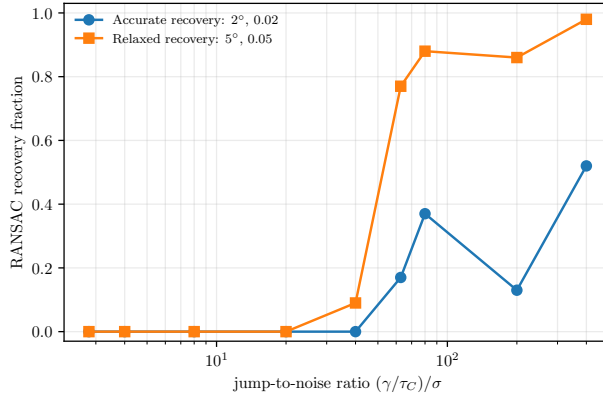


Figure 7: Empirical PP04 switching-plane recovery fraction as the discontinuity coefficient γ is varied at fixed observation-noise level $\sigma = 0.05$. For each value of γ , the experiment generates 5,000 trajectories and evaluates 100 RANSAC seeds with 20,000 proposals per seed. Initial conditions, forcing phases, and standardised observation noise are paired across the values of γ . The two curves report the fraction of seeds satisfying the accurate criterion (angle below 2° and offset error below 0.02) and the relaxed criterion (angle below 5° and offset error below 0.05), respectively, against $(\gamma/\tau_C)/\sigma$.

5.5 Trajectory-based learning of the geometry and the dynamics

The following experiments evaluate the problem of learning the dynamics from trajectory data after selecting a switching geometry. In some runs, the true switching set is supplied to the model. In the remaining runs, it is recovered from warm-up trajectories and then kept fixed during training. This separation is useful because geometry errors and vector field errors affect the learnt flow in different ways. When we use the trajectory rollout-loss to train our piecewise-smooth neural networks, we weigh the contributions to the MSE loss coming from the two sides of the switch so they are balanced. The balancing is based on the sign of the switching function at the trajectory initial condition.

5.5.1 Dry-friction oscillators

The dry-friction examples, see Equation (16), provide a controlled setting in which the true switching set is simple, but the long-time behaviour is sensitive to both the discontinuity and the sliding dynamics. The observed trajectories are

perturbed with additive Gaussian noise with $\sigma \in \{0, 10^{-2}, 5 \cdot 10^{-2}\}$. When the second-order structure is enforced, we only parametrise with the network the second component of the vector field, and force the first to be equal to p . For each case and noise level, we train the four combinations assuming the knowledge, or lack thereof, of the switching hyperplane and second-order nature of the dynamics. The forcing frequency is assumed known in the main dry-friction tables. Appendix C.2 shows that replacing the known harmonic features with fixed random Fourier features substantially degrades both local accuracy and temporal extrapolation. All dry-friction models use one hidden layer of width 10 in Φ and three hidden layers of width 10 in Ψ . In the learned-switch runs, we retain the largest 2% of the derivative-jump scores globally, augment each retained interval with its neighbours within radius 1, and use both interval endpoints as candidates. We fit one switching line using 20,000 RANSAC proposals with distance tolerance 0.03, followed by total least-squares refinement on the inliers. All models minimise (12). The velocities are estimated with a Savitzky–Golay window of 11 and polynomial degree 3. The loss weights are $\lambda_{\text{traj}} = 1$, $\lambda_{\text{vf}} = 10^{-2}$, $\lambda_{\text{nor}} = 3 \cdot 10^{-3}$, and $\lambda_{\text{tan}} = 10^{-2}$. The velocity loss excludes $|s(x)| < 0.05$, and the normal and tangential thresholds are $\tau_{\text{nor}} = 5 \cdot 10^{-3}$ and $\tau_{\text{tan}} = 10^{-3}$.

Training uses classical RK4, not the Filippov solver. This choice is based on empirical evidence of comparable performance but at a much lower time cost. The rollout horizon follows a curriculum from 2 steps to the full 10-step window. Adam is run for 500 epochs with batch size 128, weight decay 10^{-5} , gradient-norm clipping at 1, and a one-cycle schedule between $5 \cdot 10^{-4}$ and $5 \cdot 10^{-3}$. Full-horizon training and validation losses are measured every five epochs and after the final epoch, and the checkpoint with the smallest validation rollout loss is restored. The Filippov solver, see Appendix D, is used for rollout evaluation of the discontinuous models.

The recovered line is fixed during dynamics learning. In the reported tables, the column “2nd order” indicates whether the model is constrained to respect $\dot{q} = p$, so that only the acceleration component is learnt.

Table 1: Dynamics-learning results for dry-friction case 1. Each sub-table corresponds to a different trajectory-noise level.

(a) Clean data, $\sigma = 0$							
Known switch	2nd order	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	no	0.225	$3.78 \cdot 10^{-3}$	0.0348	0.0701	0.0146	$3.00 \cdot 10^{-3}$
no	yes	0.225	$3.78 \cdot 10^{-3}$	$3.90 \cdot 10^{-3}$	$5.17 \cdot 10^{-3}$	$6.27 \cdot 10^{-3}$	$4.65 \cdot 10^{-3}$
yes	no	0	0	0.0356	0.0719	0.0144	$2.86 \cdot 10^{-3}$
yes	yes	0	0	$3.18 \cdot 10^{-3}$	$3.99 \cdot 10^{-3}$	$6.41 \cdot 10^{-3}$	$4.73 \cdot 10^{-3}$
(b) Mild noise, $\sigma = 10^{-2}$							
Known switch	2nd order	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	no	0.284	$4.92 \cdot 10^{-3}$	0.0882	0.156	0.0618	0.0613
no	yes	0.284	$4.92 \cdot 10^{-3}$	0.0150	0.0235	0.0202	0.0231
yes	no	0	0	0.0887	0.157	0.0623	0.0620
yes	yes	0	0	0.0219	0.0351	0.0291	0.0315
(c) High noise, $\sigma = 5 \cdot 10^{-2}$							
Known switch	2nd order	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	no	1.34	0.0475	0.202	0.331	0.227	0.553
no	yes	1.34	0.0475	0.0869	0.129	0.164	0.514
yes	no	0	0	0.275	0.446	0.259	0.276
yes	yes	0	0	0.125	0.184	0.193	0.220

Results Table 1 and Table 2 show that the second-order constraint improves rollout and final-state RMSE in every case, noise level, and geometry setting. At $\sigma \leq 10^{-2}$, the recovered switches remain accurate, with angle errors below 0.30° and offset errors below $5 \cdot 10^{-3}$. At $\sigma = 5 \cdot 10^{-2}$, the angle errors increase to approximately 1.3° and the switching-band errors increase substantially. Known geometry does not uniformly improve rollout error. The learned-switch and known-switch models are optimised independently, and the recovered geometry can act as a slightly different inductive bias. The consistent result is the benefit of second-order structure and the degradation of local switching-band accuracy when high-noise geometry recovery deteriorates.

Table 2: Dynamics-learning results for dry-friction case 2. Each sub-table corresponds to a different trajectory-noise level.

(a) Clean data, $\sigma = 0$

Known switch	2nd order	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	no	0.194	$9.58 \cdot 10^{-4}$	0.0234	0.0537	0.0146	$3.18 \cdot 10^{-3}$
no	yes	0.194	$9.58 \cdot 10^{-4}$	$2.16 \cdot 10^{-3}$	$3.16 \cdot 10^{-3}$	$6.25 \cdot 10^{-3}$	$4.70 \cdot 10^{-3}$
yes	no	0	0	0.0236	0.0543	0.0144	$2.93 \cdot 10^{-3}$
yes	yes	0	0	$2.17 \cdot 10^{-3}$	$3.40 \cdot 10^{-3}$	$5.89 \cdot 10^{-3}$	$4.61 \cdot 10^{-3}$

(b) Mild noise, $\sigma = 10^{-2}$

Known switch	2nd order	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	no	0.300	$2.54 \cdot 10^{-3}$	0.0748	0.138	0.0617	0.0613
no	yes	0.300	$2.54 \cdot 10^{-3}$	0.0214	0.0343	0.0286	0.0311
yes	no	0	0	0.0752	0.139	0.0622	0.0618
yes	yes	0	0	0.0224	0.0356	0.0294	0.0318

(c) High noise, $\sigma = 5 \cdot 10^{-2}$

Known switch	2nd order	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	no	1.29	0.0332	0.170	0.293	0.191	0.514
no	yes	1.29	0.0332	0.131	0.196	0.207	0.484
yes	no	0	0	0.200	0.335	0.195	0.180
yes	yes	0	0	0.126	0.186	0.195	0.222

5.5.2 PP04

We use PP04, see Equation (17), as a second trajectory-learning benchmark after the dry-friction examples. The switching geometry must either be supplied to the model or recovered from trajectory data, and the learned vector field is then assessed through both local vector field errors and finite-horizon rollouts.

The forcing frequency ω is assumed known in all PP04 runs. The observed trajectories have initial conditions sampled in $[0, 1]^3$ and perturbed by additive Gaussian noise with standard deviation $\sigma \in \{0, 10^{-2}\}$. We choose these noise levels because of the analysis in Section 4.1. All PP04 models use one hidden layer of width 11 in Φ and one hidden layer of width 14 in Ψ . In the learned-switch runs, we retain the largest 0.5% of derivative-jump scores globally, augment each retained interval with its neighbours within radius 2, and use both interval endpoints as candidates. We fit one switching plane using 20,000 RANSAC proposals with distance tolerance 10^{-2} , followed by total least-squares refinement on the inliers.

Dynamics training minimises (12) using classical RK4, with $\lambda_{\text{traj}} = 1$, $\lambda_{\text{vf}} = 10^{-2}$, no switch-band exclusion, and $\lambda_{\text{nor}} = \lambda_{\text{tan}} = 0$. Dynamics-loss velocities use a Savitzky–Golay window of 21 and polynomial degree 3; warm-up geometry recovery instead uses window 51 and degree 2. The rollout curriculum grows from 2 to 50 steps. Adam is run for 500 epochs with batch size 128, weight decay 10^{-5} , gradient-norm clipping at 1, and a one-cycle schedule between 10^{-5} and 10^{-4} . Validation is evaluated every five epochs and after the final epoch, and the best full-horizon validation checkpoint is restored.

The recovered switching plane remains accurate at both noise levels. All four models reproduce the PP04 dynamics closely, although observation noise increases both rollout and local vector-field errors. Supplying the true switching plane reduces every reported error at each noise level, with the largest improvement occurring near the switching surface.

The initial state is shared across panels and selected from 30 deterministic candidates by minimising aggregate rollout error over $0 \leq t \leq 250$. Every candidate rollout starts at $t_0 = 0$. The figure displays the first 500 time units for readability. This selection is used only for visualisation and does not affect training, checkpoint selection, or the evaluation metrics.

Table 3: PP04 dynamics-learning results with known forcing frequency and Filippov rollout evaluation. Each sub-table fixes the noise level and compares the full learned-geometry pipeline against the known-switch setting.

(a) $\sigma = 0$						
Known switch	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	0.339	$5.18 \cdot 10^{-3}$	$2.66 \cdot 10^{-3}$	$2.67 \cdot 10^{-3}$	$6.18 \cdot 10^{-3}$	0.0191
yes	0	0	$1.45 \cdot 10^{-3}$	$1.7 \cdot 10^{-3}$	$1.3 \cdot 10^{-3}$	$2.19 \cdot 10^{-3}$

(b) $\sigma = 10^{-2}$						
Known switch	Switch angle	Switch offset	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
no	0.458	$4.37 \cdot 10^{-3}$	0.0111	0.0176	$7.9 \cdot 10^{-3}$	0.0244
yes	0	0	$8.07 \cdot 10^{-3}$	0.0125	$1.68 \cdot 10^{-3}$	$2 \cdot 10^{-3}$

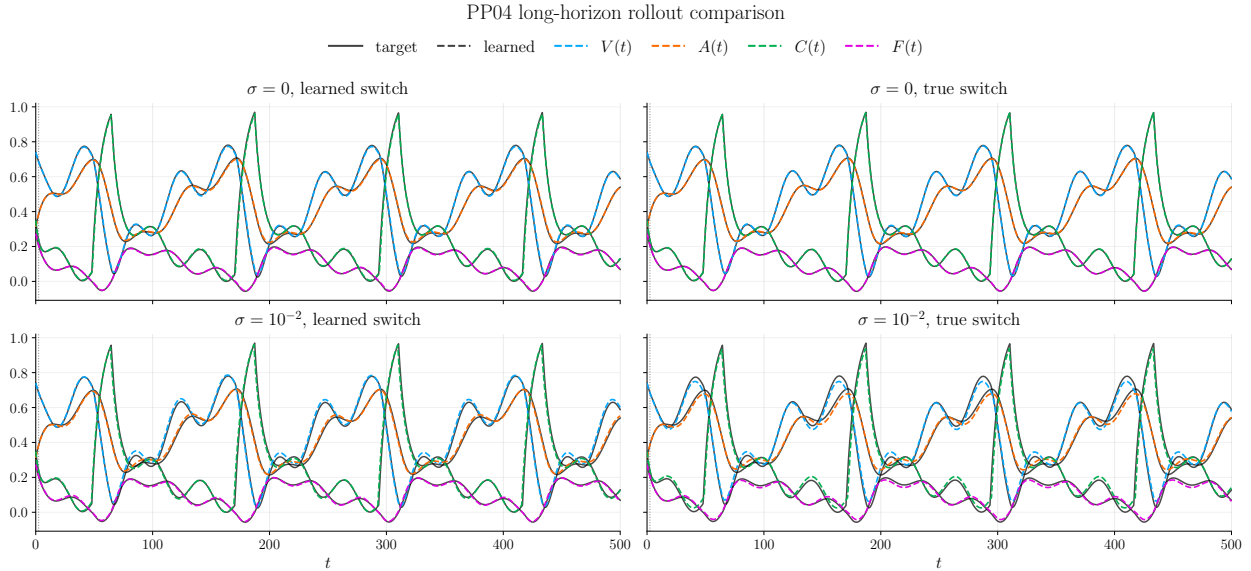


Figure 8: PP04 rollouts over $0 \leq t \leq 500$ from the shared initial state selected by minimum aggregate error across all four models, with $t_0 = 0$. Each panel corresponds to one noise and switching-geometry setting, and each trace overlays the target trajectory with the learnt trajectory for V , A , C , and the switching expression $aV - bA + d$. The PP04 state itself has three components. The clean runs are nearly indistinguishable from the target, while the noisy runs retain the recurrent structure with small local deviations.

5.5.3 Comparison with a smooth Neural ODE

We compare the structured models with a smooth Neural ODE trained on the same trajectory pools, splits, segment lengths, and known-frequency time features. The baseline is a three-hidden-layer width-64 MLP with tanh activations. It is trained for 500 epochs using Adam at the fixed learning rate 10^{-3} , weight decay 10^{-6} , batch size 128 for dry friction and for PP04, and full-horizon RK4 trajectory mean-square error. Validation is measured every five epochs and after the final epoch, and the best validation checkpoint is restored. The baseline has no switching code, geometry-recovery stage, vector-field loss, or directional regulariser. This choice follows from empirically seeing that these additional components did not improve the model performance. The trainable parameter counts are constant across noise levels and across the two dry-friction forcing cases. For dry friction, the structured Disc model has 521 trainable parameters and the smooth NODE baseline has 8705. For PP04, the corresponding counts are 448 and 8899. These totals count only parameters with gradients, not the fixed switching-hyperplane tensors.

Table 4: Comparison with a smooth Neural ODE baseline. “Disc” denotes the learned-geometry discontinuous model and “NODE” the continuous baseline. Case 1 and 2 are dry-friction oscillators. For dry friction, both the discontinuous model and the smooth Neural ODE impose the second-order relation $\dot{q} = p$ and parametrise only $\dot{p}$. For PP04, both models parametrise all three components of the vector field. “VF band” restricts vector-field error to the switching band. Bold indicates the smaller value within each matched pair. Discontinuous-model rollouts use the Filippov solver, while continuous baselines use RK4.

System	Model	Rollout RMSE	Final RMSE	VF RMSE	VF band RMSE
Case 1, $\sigma = 0$	Disc	$3.90 \cdot 10^{-3}$	$5.17 \cdot 10^{-3}$	$6.27 \cdot 10^{-3}$	$4.65 \cdot 10^{-3}$
Case 1, $\sigma = 0$	NODE	$3.53 \cdot 10^{-2}$	$5.55 \cdot 10^{-2}$	$8.65 \cdot 10^{-2}$	$4.17 \cdot 10^{-1}$
Case 1, $\sigma = 10^{-2}$	Disc	$1.50 \cdot 10^{-2}$	$2.35 \cdot 10^{-2}$	$2.02 \cdot 10^{-2}$	$2.31 \cdot 10^{-2}$
Case 1, $\sigma = 10^{-2}$	NODE	$4.88 \cdot 10^{-2}$	$7.51 \cdot 10^{-2}$	$1.03 \cdot 10^{-1}$	$4.34 \cdot 10^{-1}$
Case 2, $\sigma = 0$	Disc	$2.16 \cdot 10^{-3}$	$3.16 \cdot 10^{-3}$	$6.25 \cdot 10^{-3}$	$4.70 \cdot 10^{-3}$
Case 2, $\sigma = 0$	NODE	$2.79 \cdot 10^{-2}$	$4.43 \cdot 10^{-2}$	$8.65 \cdot 10^{-2}$	$4.17 \cdot 10^{-1}$
Case 2, $\sigma = 10^{-2}$	Disc	$2.14 \cdot 10^{-2}$	$3.43 \cdot 10^{-2}$	$2.86 \cdot 10^{-2}$	$3.11 \cdot 10^{-2}$
Case 2, $\sigma = 10^{-2}$	NODE	$4.52 \cdot 10^{-2}$	$6.75 \cdot 10^{-2}$	$1.03 \cdot 10^{-1}$	$4.34 \cdot 10^{-1}$
PP04, $\sigma = 0$	Disc	$2.66 \cdot 10^{-3}$	$2.67 \cdot 10^{-3}$	$6.18 \cdot 10^{-3}$	$1.91 \cdot 10^{-2}$
PP04, $\sigma = 0$	NODE	$3.28 \cdot 10^{-2}$	$6.24 \cdot 10^{-2}$	$8.96 \cdot 10^{-3}$	$2.71 \cdot 10^{-2}$
PP04, $\sigma = 10^{-2}$	Disc	$1.11 \cdot 10^{-2}$	$1.76 \cdot 10^{-2}$	$7.9 \cdot 10^{-3}$	$2.44 \cdot 10^{-2}$
PP04, $\sigma = 10^{-2}$	NODE	$2.61 \cdot 10^{-2}$	$3.32 \cdot 10^{-2}$	$1.05 \cdot 10^{-2}$	$3.11 \cdot 10^{-2}$

The compact discontinuous model outperforms the substantially larger NODE baseline in every metric at both reported noise levels. Thus the comparison supports strong parameter efficiency and model flexibility for the considered examples.

6 Discussion and conclusion

We have proposed a two-stage framework for learning piecewise-smooth dynamical systems from trajectory data. The method first estimates affine switching geometry from derivative jumps, and then learns dynamics adapted to the regions induced by the recovered or prescribed switching set. This separation makes the model interpretable and allows prior knowledge to be incorporated either in the geometry-learning stage or in the local dynamics.

The theoretical analysis provides results for both stages. It derives a local expected detectability condition that shows how observation noise, derivative-estimation error, smoothing, and jump size interact under regularity and transversal-crossing assumptions. It also shows that the proposed geometry-preserving neural class can uniformly approximate piecewise-smooth vector fields on compact sets away from the prescribed switching hyperplanes, while confining discontinuities to the prescribed partition.

The experiments support these findings. With known geometry, the proposed architecture avoids spurious off-switch jumps. With the recovered geometry, the learned models produce accurate rollouts when the derivative-jump signal is sufficiently strong relative to the noise. The dry-friction and PP04 tests also show the value of incorporating structural information, such as second-order dynamics or known forcing frequencies. More broadly, the results show that explicitly learning and preserving switching geometry offers a practical route to interpretable models of discontinuous dynamics.

The present study is restricted to fully observed low-dimensional states, affine switching geometry, and a sequential geometry–dynamics pipeline. The Filippov-aware solver is further restricted to a single switching hyperplane.

Future work will consider experimental noisy data, higher-dimensional systems, interacting switches, and non-affine switching surfaces. Another natural direction is to combine the method with encoder–decoder models, so that discontinuous latent dynamics can be learned from videos or other high-dimensional scientific observations.

Acknowledgements

The authors would like to thank Dr Alice Cicirello for the insightful discussions at the start of this project. EJ was funded by the Knut and Alice Wallenberg Foundation grant 2024.0440. CBS acknowledges support from the Royal

Society Wolfson Fellowship, the EPSRC advanced career fellowship EP/V029428/1, the Wellcome Innovator Awards 215733/Z/19/Z and 221633/Z/20/Z. DM, CB, and CBS acknowledge the support of the EPSRC programme grant EP/V026259/1. DM and CBS acknowledge support from the EPSRC programme grant EP/Y028783/1. All the authors acknowledge support from the EU through the Marie Skłodowska-Curie Actions Staff Exchanges project REMODEL, grant agreement 101131557.

AI use declaration ChatGPT was used to assist in checking mathematical proofs and identifying notation issues and inconsistencies across the manuscript prior to submission. Codex was used to assist with the preparation of the research code, under direct and substantial supervision by the authors. The code is publicly available in the GitHub repository associated with this paper. The authors assume responsibility for all content.

References

- [1] R. C. ARKIN, *Behavior-based robotics*, Intelligent Robots and Autonomous Agents, MIT Press, Cambridge, MA, May 1998.
- [2] B. BROGLIATO, *Nonsmooth Mechanics: Models, Dynamics and Control*, Communications and Control Engineering, Springer International Publishing, 3rd ed., 2016, <https://doi.org/10.1007/978-3-319-28664-8>.
- [3] S. L. BRUNTON, J. L. PROCTOR, AND J. N. KUTZ, *Discovering governing equations from data by sparse identification of nonlinear dynamical systems*, Proceedings of the National Academy of Sciences, 113 (2016), pp. 3932–3937, <https://doi.org/10.1073/pnas.1517384113>.
- [4] C. BUDD AND K. S. MORUPISI, *Grazing bifurcations and transitions between periodic states of the PP04 model for the glacial cycle*, IMA Journal of Applied Mathematics, 87 (2022), pp. 462–491, <https://doi.org/10.1093/imamat/hxac013>.
- [5] P. CAÑIZARES, D. MURARI, C.-B. SCHÖNLIEB, F. SHERRY, AND Z. SHUMAYLOV, *Symplectic Neural Flows for Modeling and Discovery*, 2024, <https://arxiv.org/abs/2412.16787>.
- [6] E. CELLEDONI, A. LEONE, D. MURARI, AND B. OWREN, *Learning Hamiltonians of constrained mechanical systems*, Journal of Computational and Applied Mathematics, 417 (2023), p. 114608, <https://doi.org/10.1016/j.cam.2022.114608>.
- [7] K. CHAMPION, B. LUSCH, J. N. KUTZ, AND S. L. BRUNTON, *Data-driven discovery of coordinates and governing equations*, Proceedings of the National Academy of Sciences, 116 (2019), pp. 22445–22451, <https://doi.org/10.1073/pnas.1906995116>.
- [8] R. CHARTRAND, *Numerical Differentiation of Noisy, Nonsmooth Data*, ISRN Applied Mathematics, 2011 (2011), pp. 1–11, <https://doi.org/10.5402/2011/164564>.
- [9] R. T. Q. CHEN, Y. RUBANOVA, J. BETTENCOURT, AND D. DUVENAUD, *Neural Ordinary Differential Equations*, in Advances in Neural Information Processing Systems, vol. 31, 2018, pp. 6572–6583.
- [10] M. CRANMER, *Interpretable machine learning for science with PySR and SymbolicRegression.jl*, 2023, <https://arxiv.org/abs/2305.01582>.
- [11] M. CRANMER, S. GREYDANUS, S. HOYER, P. BATTAGLIA, D. SPERGEL, AND S. HO, *Lagrangian Neural Networks*, 2020, <https://arxiv.org/abs/2003.04630>.
- [12] M. CRANMER, A. SANCHEZ-GONZALEZ, P. BATTAGLIA, R. XU, K. CRANMER, D. SPERGEL, AND S. HO, *Discovering Symbolic Models from Deep Learning with Inductive Biases*, in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 17429–17442.
- [13] F. DELLA SANTA AND S. PIERACCINI, *Discontinuous neural networks and discontinuity learning*, Journal of Computational and Applied Mathematics, 419 (2023), p. 114678, <https://doi.org/10.1016/j.cam.2022.114678>.
- [14] M. DI BERNARDO, C. J. BUDD, A. R. CHAMPNEYS, AND P. KOWALCZYK, *Piecewise-smooth Dynamical Systems: Theory and Applications*, Applied mathematical sciences; 163, Springer-Verlag, London, 2008, <https://doi.org/10.1007/978-1-84628-708-4>.
- [15] C. EDWARDS AND S. SPURGEON, *Sliding Mode Control: Theory And Applications*, CRC Press, Aug. 1998, <https://doi.org/10.1201/9781498701822>.
- [16] A. F. FILIPPOV, *Differential Equations with Discontinuous Righthand Sides*, vol. 18 of Mathematics and Its Applications (Soviet Series), Kluwer Academic Publishers Group, Dordrecht, 1988. Translated from the Russian.

- [17] M. A. FISCHLER AND R. C. BOLLES, *Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography*, Communications of the ACM, 24 (1981), pp. 381–395, <https://doi.org/10.1145/358669.358692>.
- [18] S. GREYDANUS, M. DZAMBA, AND J. YOSINSKI, *Hamiltonian neural networks*, in Advances in Neural Information Processing Systems, vol. 32, 2019, pp. 15353–15363.
- [19] S. HOCHREITER AND J. SCHMIDHUBER, *Long short-term memory*, Neural Computation, 9 (1997), pp. 1735–1780, <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [20] M. R. JEFFREY, *Modeling with Nonsmooth Dynamics*, vol. 7 of Frontiers in applied dynamical systems: reviews and tutorials, Springer, United States, Feb. 2020, <https://doi.org/10.1007/978-3-030-35987-4>.
- [21] P. JIN, Z. ZHANG, A. ZHU, Y. TANG, AND G. E. KARNIADAKIS, *SympNets: Intrinsic structure-preserving symplectic networks for identifying hamiltonian systems*, Neural Networks, 132 (2020), pp. 166–179, <https://doi.org/10.1016/j.neunet.2020.08.017>.
- [22] D. KARNOPP, *Computer simulation of stick-slip friction in mechanical dynamic systems*, Journal of Dynamic Systems, Measurement, and Control, 107 (1985), pp. 100–103, <https://doi.org/10.1115/1.3140698>.
- [23] P. KIDGER, J. MORRILL, J. FOSTER, AND T. LYONS, *Neural controlled differential equations for irregular time series*, in Advances in Neural Information Processing Systems, vol. 33, 2020, pp. 6696–6707.
- [24] I. KONG, J. CHEN, S. LANGER, AND J. SCHMIDT-HIEBER, *On the Expressivity of Deep Heaviside Networks*, Constructive Approximation, (2026), <https://doi.org/10.1007/s00365-026-09766-3>.
- [25] C. LATHOURAKIS AND A. CICIRELLO, *Physics enhanced sparse identification of dynamical systems with discontinuous nonlinearities*, Nonlinear Dynamics, 112 (2024), pp. 11237–11264, <https://doi.org/10.1007/s11071-024-09652-2>.
- [26] B. LUSCH, J. N. KUTZ, AND S. L. BRUNTON, *Deep learning for universal linear embeddings of nonlinear dynamics*, Nature Communications, 9 (2018), p. 4950, <https://doi.org/10.1038/s41467-018-07210-0>.
- [27] K. S. MORUPISI AND C. BUDD, *An analysis of the periodically forced PP04 climate model, using the theory of non-smooth dynamical systems*, IMA Journal of Applied Mathematics, 86 (2021), pp. 76–120, <https://doi.org/10.1093/imamat/hxaa039>.
- [28] D. PAILLARD AND F. PARRENIN, *The antarctic ice sheet and the triggering of deglaciations*, Earth and Planetary Science Letters, 227 (2004), pp. 263–271, <https://doi.org/10.1016/j.epsl.2004.08.023>.
- [29] P. PETERSEN AND J. ZECH, *Mathematical theory of deep learning*, arXiv preprint arXiv:2407.18384, (2024).
- [30] C. RACKAUCKAS, Y. MA, J. MARTENSEN, C. WARNER, K. ZUBOV, R. SUPEKAR, D. SKINNER, A. RAMADHAN, AND A. EDELMAN, *Universal differential equations for scientific machine learning*, 2020, <https://arxiv.org/abs/2001.04385>.
- [31] Y. RUBANOVA, R. T. Q. CHEN, AND D. DUVENAUD, *Latent ordinary differential equations for irregularly-sampled time series*, in Advances in Neural Information Processing Systems, vol. 32, 2019.
- [32] A. SAVITZKY AND M. J. E. GOLAY, *Smoothing and differentiation of data by simplified least squares procedures*, Analytical Chemistry, 36 (1964), pp. 1627–1639, <https://doi.org/10.1021/ac60214a047>.
- [33] F. VAN BREUGEL, J. N. KUTZ, AND B. W. BRUNTON, *Numerical differentiation of noisy data: A unifying multi-objective optimization framework*, IEEE Access, 8 (2020), pp. 196865–196877, <https://doi.org/10.1109/access.2020.3034077>.
- [34] P. R. VLACHAS, W. BYEON, Z. Y. WANLESS, T. P. SAPSIS, AND P. KOUMOUTSAKOS, *Data-driven forecasting of high-dimensional chaotic systems with long short-term memory networks*, Proceedings of the Royal Society A: Mathematical, Physical and Engineering Sciences, 474 (2018), p. 20170844, <https://doi.org/10.1098/rspa.2017.0844>.
- [35] D. YAROTSKY, *Error bounds for approximations with deep ReLU networks*, Neural Networks, 94 (2017), pp. 103–114.

A Fitting linear models in the presence of outliers

Out of a sample consisting of M total points, with M_i inliers and M_o outliers, RANSAC is a randomised algorithm that iteratively separates inliers from outliers. It works by randomly selecting d points to determine a $(d - 1)$ -dimensional affine subspace of $\mathbb{R}^d$. Then, a hyperplane is fitted to these points, and the points sufficiently close (say, within a distance ε) are included in the *consensus set*. A model is deemed as reasonable if the consensus set is sufficiently large. The algorithm is repeated for a fixed number of iterations, each time keeping the model with the largest consensus set.

The number of inliers selected follows a hypergeometric distribution (as there is no replacement), so the probability r of selecting exactly d inliers is given by

$$r = \frac{\binom{M_i}{d}}{\binom{M}{d}}.$$

The probability of selecting at least one point which is not an inlier is therefore $1 - r$, and the probability of this happening k times is thus $(1 - r)^k$, which is the probability that the RANSAC algorithm has not found a good model in k iterations. The probability that at least one iteration has included only inliers is therefore $p = 1 - (1 - r)^k$. Thus, to ensure that RANSAC has reached the desired accuracy in k iterations with probability p , we must take

$$k \geq \frac{\log(1 - p)}{\log(1 - r)}.$$

We see that we must estimate r , meaning we need to determine, or at least bound, the number of points included as candidates for the RANSAC algorithm, that is, M and M_i .

Algorithm 2 describes the hyperplane-fitting step in the geometry learning phase.

Algorithm 2 RANSAC($\mathcal{C}, \varepsilon, M, m_{\text{req}}$)

Require: Candidate cloud $\mathcal{C} \subset \mathbb{R}^d$; tolerance ε ; iterations M ; required support m_{req} .

Ensure: Best-supported hyperplane proposal and its consensus set, or failure.

```

1: Initialise the current winner as empty.
2: for  $m = 1, \dots, M$  do
3:   Sample  $d$  points uniformly without replacement from  $\mathcal{C}$ .
4:   Construct the hyperplane  $\Gamma_m = \{x \in \mathbb{R}^d : a_m^\top x + b_m = 0\}$  through the sampled points.
5:   if the sampled points are degenerate then
6:     continue.
7:   end if
8:   Compute the consensus set  $\mathcal{I}_m = \{x \in \mathcal{C} : \text{dist}(x, \Gamma_m) \leq \varepsilon\}$ .
9:   if  $|\mathcal{I}_m| < m_{\text{req}}$  then
10:    continue.
11:  end if
12:  Compute the mean inlier distance

```

$$D_m = \frac{1}{|\mathcal{I}_m|} \sum_{x \in \mathcal{I}_m} \text{dist}(x, \Gamma_m).$$

```

13:   Replace the winner if  $(|\mathcal{I}_m|, -D_m)$  is lexicographically larger than its current score.
14: end for
15: if no winner was found then
16:   fail.
17: end if
18: return  $(a_\star, b_\star, \mathcal{I}_\star)$ .

```

B Additional proofs for the universality result

Proof of Lemma 4.1. The arguments in this proof apply indistinguishably to the elements in $\mathcal{F}_{\text{ReLU}}^{d_1, d_2}$ and $\mathcal{F}_{\text{ReLU}_J}^{d_1, d_2}$. Scalar multiplication and post-composition by affine maps are immediate. For addition and parallel concatenation, one first pads the shallower network with identity layers of the form (14) so that the two networks have the same depth. One may then use block-diagonal affine maps and componentwise activations to run the two networks in parallel. A final affine map gives either the concatenation or the sum. $\square$

Proof of Proposition 4.3. Let $f, g \in \mathcal{F}_P^d$. By definition there are dimensions $r_f, r_g \in \mathbb{N}$, jump networks $\Phi_f \in \mathcal{F}_{\text{ReLU}_J}^{d_P, r_f}$ and $\Phi_g \in \mathcal{F}_{\text{ReLU}_J}^{d_P, r_g}$, and continuous networks $\Psi_f \in \mathcal{F}_{\text{ReLU}}^{d+r_f+1, d}$ and $\Psi_g \in \mathcal{F}_{\text{ReLU}}^{d+r_g+1, d}$ such that

$$f(x, t) = \Psi_f(x, t, \Phi_f(z_P(x))), \quad g(x, t) = \Psi_g(x, t, \Phi_g(z_P(x))).$$

By Lemma 4.1, the parallel concatenation $\Phi(z) := [\Phi_f(z), \Phi_g(z)] \in \mathcal{F}_{\text{ReLU}_J}^{d_P, r_f+r_g}$ and the parallel concatenation of Ψ_f and Ψ_g both belong to the corresponding ReLU classes. A final affine map produces any linear combination of f and g . Hence $\mathcal{F}_P^d$ is a vector space. $\square$

C Additional experiments for the dry friction system

C.1 Representative dry-friction rollouts

The aggregate errors in Tables 1 and 2 summarise performance over the evaluation set. Here we show representative long-horizon trajectories to illustrate the qualitative behaviour behind those numbers. We fix the initial condition $(q_0, p_0) = (0.1, 0.1)$ and $t_0 = 0$ for all runs. All panels use the second-order model, known forcing frequency, noise level $\sigma = 10^{-2}$, and the Filippov-aware rollout solver. This noise level is useful for visualisation because it is large enough to make geometry recovery nontrivial, but not so large that the recovered-switch model is dominated by geometry failure.

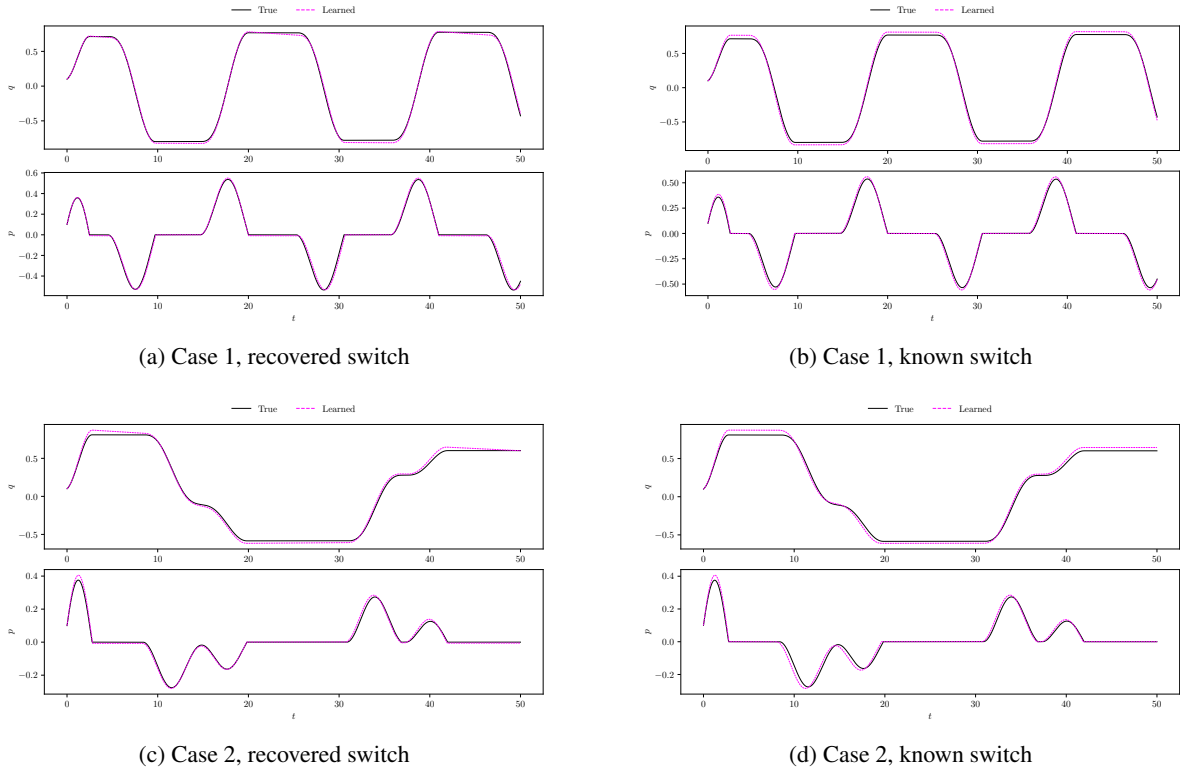


Figure 9: Representative long-horizon dry-friction rollouts at $\sigma = 10^{-2}$. Each panel compares the target trajectory with the learned trajectory for q and p . The left column shows the full pipeline, where the switching line is recovered from warm-up trajectories. The right column isolates the dynamics learning by supplying the true switching line.

The known-switch panels show that, once the switching line is supplied, the second-order model accurately reproduces both the smooth portions and the sliding intervals where $p = 0$. The recovered-switch panels remain close to the target trajectories, but small discrepancies appear near transitions into and out of sliding. This is consistent with the main tables: the rollout errors remain small in the mild-noise regime, while the banded vector-field errors are more sensitive to small errors in the recovered switching line. Thus, the qualitative degradation in the full pipeline is mainly a local switching-geometry effect rather than a global instability of the learned dynamics.

Table 5: Effect of removing the known forcing frequency in the dry-friction experiments. All rows use the true switching manifold and the second-order model. Known/Unknown refer to known and unknown forcing frequency ω . Bold entries are the lowest between known and unknown frequency.

Setting	Rollout RMSE		Final RMSE		VF RMSE	
	Known	Unknown	Known	Unknown	Known	Unknown
case 1, $\sigma = 0$	$3.18 \cdot 10^{-3}$	$2.12 \cdot 10^{-2}$	$3.99 \cdot 10^{-3}$	$3.86 \cdot 10^{-2}$	$6.41 \cdot 10^{-3}$	$3.47 \cdot 10^{-2}$
case 1, $\sigma = 0.01$	$2.19 \cdot 10^{-2}$	$3.47 \cdot 10^{-2}$	$3.51 \cdot 10^{-2}$	$5.22 \cdot 10^{-2}$	$2.91 \cdot 10^{-2}$	$5.74 \cdot 10^{-2}$
case 1, $\sigma = 0.05$	$1.25 \cdot 10^{-1}$	$1.78 \cdot 10^{-1}$	$1.84 \cdot 10^{-1}$	$2.55 \cdot 10^{-1}$	$1.93 \cdot 10^{-1}$	$4.15 \cdot 10^{-1}$
case 2, $\sigma = 0$	$2.17 \cdot 10^{-3}$	$1.69 \cdot 10^{-2}$	$3.40 \cdot 10^{-3}$	$2.83 \cdot 10^{-2}$	$5.89 \cdot 10^{-3}$	$4.05 \cdot 10^{-2}$
case 2, $\sigma = 0.01$	$2.24 \cdot 10^{-2}$	$5.09 \cdot 10^{-2}$	$3.56 \cdot 10^{-2}$	$8.04 \cdot 10^{-2}$	$2.94 \cdot 10^{-2}$	$7.39 \cdot 10^{-2}$
case 2, $\sigma = 0.05$	$1.26 \cdot 10^{-1}$	$2.07 \cdot 10^{-1}$	$1.86 \cdot 10^{-1}$	$3.18 \cdot 10^{-1}$	$1.95 \cdot 10^{-1}$	$4.47 \cdot 10^{-1}$

C.2 Unknown forcing frequency

The dry-friction tables in the main body of the paper assume that the forcing frequency is known. To test the importance of this assumption, we repeat the strongest dry-friction configuration, namely the true switching manifold with the second-order model, but remove the known harmonic time features. The model receives random Fourier time features. The training setup, noise levels, architecture, and Filippov rollout evaluation are otherwise the same as in the main dry-friction experiments.

Table 5 shows that removing the known forcing frequency consistently degrades temporal extrapolation and vector-field recovery. A representative low-noise comparison is shown in Figure 10. We fix the initial condition $(q_0, p_0) = (0.1, 0.1)$ and $t_0 = 0$ for both runs.

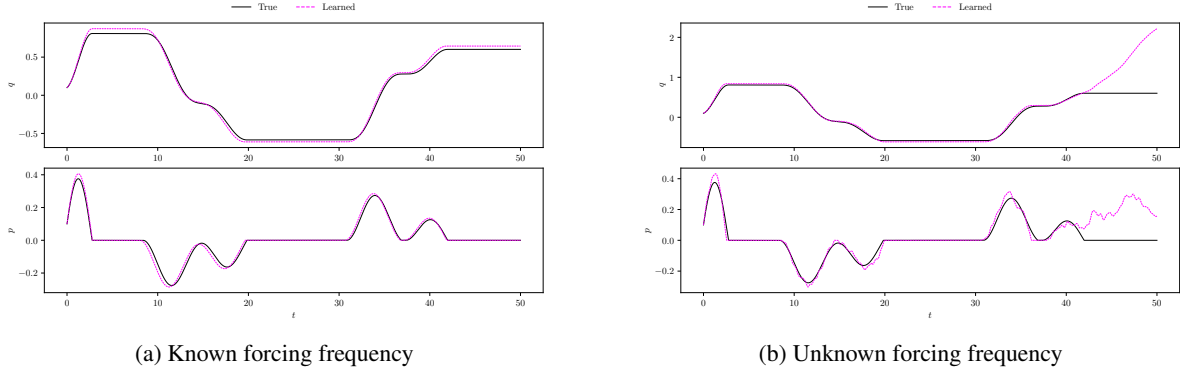


Figure 10: Representative long-horizon trajectories for dry-friction case 2 with $\sigma = 10^{-2}$, true switching geometry, and second-order structure. The left panel uses the known harmonic forcing frequency, while the right panel replaces it with random Fourier time features.

D Filippov-aware trajectory solver

This appendix describes the event-aware solver used for the rollout evaluations of the discontinuous models. The solver is used for single-hyperplane discontinuous models with switching function $s(x) = a^\top x + b$, and with learned one-sided vector fields $f^+(x, t)$ and $f^-(x, t)$. The purpose of the solver is to avoid treating a crossing of $\Sigma := \{x : s(x) = 0\}$ as an ordinary smooth step, and to evolve trajectories along Σ when the two one-sided fields generate sliding.

For a step from x_n to x_{n+1} with step size h , we first compute the side $r_n = \text{sign}(s(x_n))$. A trial step is then taken with a branch-frozen second-order Runge–Kutta update, using f^+ throughout the step if $r_n = +1$ and f^- throughout the step if $r_n = -1$. Let $\tilde{x}_{n+1}$ denote this trial endpoint. A switching event is detected when $s(x_n) s(\tilde{x}_{n+1}) < 0$. We also trigger event handling when the state is already close to the manifold, $|s(x_n)| < \varepsilon$, and the one-sided normal velocities indicate an attractive sliding region. In the experiments we use a small tolerance ε only to decide when this special handling is needed.

When a crossing is detected, the event time is located by bisection on the interval $[0, h]$. During this search the same branch-frozen RK2 propagator is used, starting from x_n on the pre-crossing side. After bisection, the event point is

projected back onto the hyperplane,

$$x_{\Sigma} = x_{\text{event}} - s(x_{\text{event}})a/\|a\|_2^2,$$

which removes the small numerical residual in the switching function.

The vector field on the switching manifold is defined using the standard Filippov convex combination. The trajectory is then advanced along Σ using small explicit substeps and projected back onto the hyperplane after each substep. If the attractive condition fails, the trajectory exits the manifold on the side indicated by the resulting one-sided normal velocities.

The event-handling procedure is repeated within one macro-step if necessary, up to a fixed small maximum number of events. If no crossing or sliding event is detected, the solver reduces to the branch-frozen RK2 update on the current side. This propagation rule is used for the long-horizon rollout diagnostics reported in the experiments. The current implementation cannot handle multi-surface interactions. The event detector is also not a fully general root-finding procedure. It may miss a crossing if a trajectory touches or crosses Σ an even number of times within a single branch-frozen trial step, so that the endpoint sign does not change. In the systems considered here, the chosen time steps were small enough that this event-detection rule was sufficient in practice. For more details on the implementation, see the GitHub repository <https://github.com/davidemurari/learn-piecewise-smooth>.

E Sequential recovery of multiple switching lines

The geometry-recovery routine used in the main experiments is designed to find a single affine switching set from a cloud of candidate points. This appendix records a simple extension of the same idea to a setting with more than one interface. For this example, the data are noisy samples from three non-parallel lines in $\mathbb{R}^2$, plus a small number of outliers. The purpose is to test whether the RANSAC–TLS step can be applied sequentially.

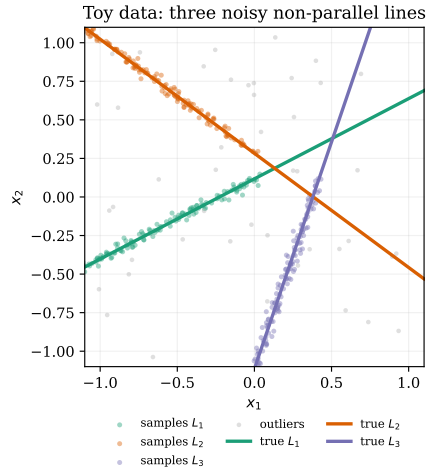


Figure 11: Toy candidate cloud used in the multi-line recovery test. The coloured points are noisy samples from the three true non-parallel lines, and the grey points are uniform outliers.

Figure 11 shows the toy data. Each coloured cloud is sampled near one true line, with independent Gaussian perturbations. The grey points are unrelated outliers. For a line written as $a^\top x + b = 0$, $\|a\|_2 = 1$, we run the same RANSAC routine used in the geometry-learning code. After one line has been recovered, its inliers are removed from the cloud, and the procedure is repeated on the remaining points. We show in Figure 12 the number of inliers of each attempt. This figure demonstrates that when the number of hyperplanes is unknown, the number of inliers can be a viable heuristic for determining it.

Figure 13 illustrates the three passes. The first pass matches L_2 with angle error 0.0854° and offset error $2.55 \cdot 10^{-4}$. The second pass matches L_1 with angle error 0.135° and offset error $1.35 \cdot 10^{-3}$. The third pass matches L_3 with angle error 0.111° and offset error $4.82 \cdot 10^{-3}$.

This toy problem also shows the main limitation of the sequential strategy. The removal step is greedy. If the first recovered line absorbs points belonging to another nearby line, later passes inherit that mistake. Thus, this is not a complete multi-surface estimator or a full reconstruction method for polygonal cell complexes. It is a useful

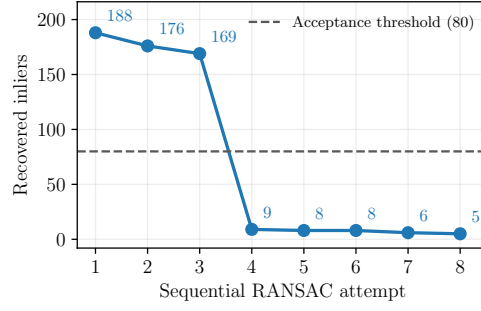


Figure 12: Number of inliers for each RANSAC attempt, after removing the previous inliers.

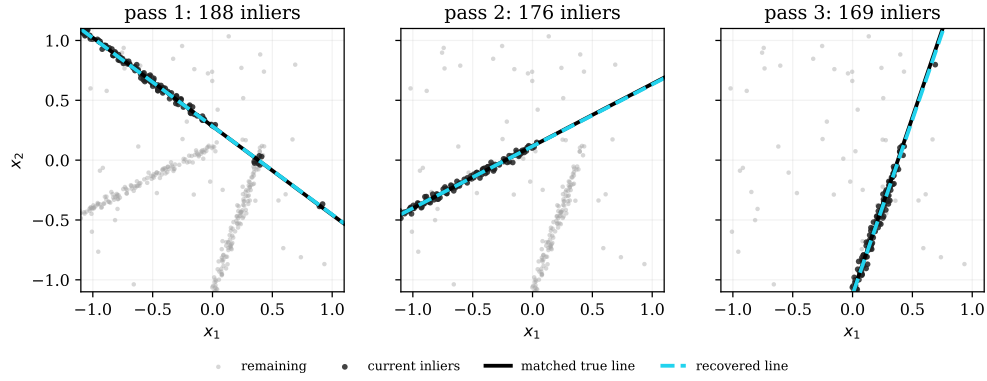


Figure 13: Sequential RANSAC–TLS extraction. In each panel, black points are the inliers selected during that pass, while light grey points remain available for later passes; points removed during earlier passes are omitted. The solid black line is the matched true line, and the dashed cyan line is the recovered TLS-refined estimate.

diagnostic for well-separated affine interfaces, but genuinely interacting or too close switching sets would require a joint model-selection step rather than repeated single-line recovery.